

RESEARCH ARTICLE

Optimizing Test Case Prioritization With Meta Deep Reinforcement Learning in Continuous Integration

NAHLAH A. ALRAKBAN^{1,2}, MUBARAK ALRASHOUD^{1,3},
AND M. ABDULLAH-AL-WADUD¹, (Member, IEEE)

¹Department of Software Engineering, College of Computer and Information Sciences, King Saud University, Riyadh 11543, Saudi Arabia

²Department of Computer Science, College of Computing and Informatics, Saudi Electronic University, Riyadh 11673, Saudi Arabia

³Chair of Smart Technologies, Department of Software Engineering, College of Computer and Information Sciences, King Saud University, Riyadh 11543, Saudi Arabia.

Corresponding author: Nahlah A. AlRakban (n.alrakban@seu.edu.sa)

This work was supported by Ongoing Research Funding Program - Research Chairs, King Saud University, Riyadh, Saudi Arabia, under Grant ORF-RC-2025-2000.

ABSTRACT Software developers use Continuous Integration (CI) environments to reduce integration issues and expedite development cycles. Regression testing is an important part of CI practice, as it includes reconducting all test cases to guarantee system stability after upgrades. However, as test suites grow, this process becomes increasingly resource-intensive and time-consuming. While many Test Case Prioritization (TCP) techniques have been proposed to address this challenge, previous approaches often rely on static configurations and lack the adaptability needed to handle the dynamic nature of CI environments and different dataset complexities. To address these gaps, this study presents a novel TCP framework based on Deep Reinforcement Learning (DRL), integrating a pairwise ranking model with state-of-the-art DRL algorithms, including A2C, DQN, PPO, and TRPO. The proposed framework improves prioritization accuracy and execution efficiency, particularly when combined with an optimal cycle count strategy. An adaptive training framework based on Meta-Deep Reinforcement Learning (meta-DRL) was introduced to further enhance the adaptability of the framework. This component allows the DRL agent to assess its performance during training and dynamically modify the essential hyperparameters, thereby enhancing its ability to develop successful prioritizing methods over time. Finally, the results of the proposed methodology demonstrate that Meta-Deep Reinforcement Learning (meta-DRL) significantly reduces the training time and achieves a 60% reduction compared to existing approaches. These findings show the efficiency of meta-DRL-based TCP in providing a scalable and adaptive solution for enhancing regression testing in CI environments.

INDEX TERMS Continuous integration, deep reinforcement learning, meta-DRL, reward function, test case prioritization.

I. INTRODUCTION

The software development industry is rapidly increasing and scaling. Rapid changes in hardware and software infrastructures demand a reliable and robust solution. With changes occurring in the development of software code, there is always a need to integrate these changes efficiently. Continuous Integration (CI) is a widely used approach in software development in which developers regularly integrate

their work. This process encompasses numerous activities wherein the software undergoes modifications, construction, and assessment through regression testing. As new software systems become more complex, the test suite tends to develop increasingly larger and more expensive [1]. Regression testing is a vital part of the software development process [2], which involves running test cases over again to ensure that any new modifications or updates made to the system do not result in the disruption of the software testing firm; the process of managing and implementing a significant quantity of test cases can be both time-consuming and

The associate editor coordinating the review of this manuscript and approving it for publication was Porfirio Tramontana¹.

resource-intensive [3]. In 2017, 606 software flaws resulted in an estimated economic loss of \$1.7 trillion, impacting nearly 3.7 billion individuals worldwide [3]. To address this issue, numerous researchers and practitioners have been working to devise effective strategies for enhancing regression testing within the CI environment, aiming to augment software reliability. The enhancement of regression testing depends fundamentally on three principal strategies: Test Case Selection (TCS) [4], [5], Test Suite Minimization (TSM) [4], [6], and Test Case Prioritization (TCP) [4], [7]. The **TCS** technique involves the selection of specific test cases that align with corrected sections in the System Under Test (SUT) [8]. Nevertheless, the primary concept of the TCS strategy is to select an extensive set of test cases that optimizes coverage for various features within the software system. The **TSM** strategy emphasizes the minimization of size and duplication within the test suite, differentiates between repeating experiments, and removes test cases from a test suite execution to intentionally decrease the overall number of tests that must be performed [8]. The objective of the **TCP** strategy is to arrange a collection of test cases in a specific order to achieve an early enhancement according to the defined criteria. This strategy prioritizes and executes test cases based on specific requirements to achieve the desired results, such as detecting faults at an early stage and providing developers with feedback [7], [8]. Moreover, TCP enables the ongoing execution of different tests until predetermined time or budget constraints are reached by executing the test suite according to the set prioritizing order. Furthermore, the TCP strategy offers a distinct advantage in comparison to TCS and TSM because it retains all test cases included in the execution process [8]. Test case prioritization presents two concerns for test professionals [9]. First, it is important to prioritize test cases with a higher probability of detecting regression faults. Second, the time required for testing was reduced. The challenges provided by TCP techniques have proven to be difficult to address using only search or heuristics-based algorithms [10]. Therefore, other techniques, such as machine learning (ML), have been utilized. Several ML techniques have been offered as potential solutions to address the issues associated with test case prioritization. However, the current machine learning (ML)-based solutions for test case prioritization in a CI environment are constrained by the substantial volume of test execution data generated through testing, which requires enhancement to effectively address the dynamic nature of the CI environment [10].

To address these challenges, Deep Reinforcement Learning (DRL) algorithms have emerged as the most appropriate approach for prioritizing test cases in a CI context, owing to their abilities compared to alternative machine learning techniques. DRL has successfully automated complex testing tasks such as robot automation, gaming, traffic control, image processing, and test case prioritization, which enable continuous adaptation [3]. Currently available DRL-based TCP methodologies generally employ static training parameters, such as fixed cycle counts and extensive training episodes,

to achieve competitive performance, resulting in increased execution time, significant computational overhead, and limited adaptability to diverse CI datasets.

This study introduces a novel DRL-based TCP framework designed to address existing gaps. It integrates a pairwise ranking model alongside advanced DRL algorithms, including A2C, DQN, PPO, and TRPO, while employing an optimal cycle count strategy. Each algorithm was applied independently across all datasets, ensuring a direct and fair comparison of performance. This framework enables adaptive prioritization based on dataset complexity while minimizing the training overhead. Initial experiments with this strategy demonstrated that identifying an optimal cycle count markedly enhanced both execution efficiency and prioritization accuracy. The encouraging results prompted the advancement of the methodology by incorporating an adaptive training framework utilizing meta-DRL. This framework allows the system to independently modify essential training parameters such as the cycle count, learning rate, discount factor, and number of episodes, according to real-time performance feedback. Instead of depending on fixed, manually adjusted settings, meta-DRL extends the cycle count method into a self-adaptive, learning-to-learn mechanism that is more appropriate for the dynamic characteristics of CI environments. The experimental findings indicate that the proposed meta-DRL architecture consistently achieves excellent prioritization accuracy, as reflected by the enhanced APFD and NRPA metrics in both the simple and enriched CI datasets. Significantly, meta-DRL not only improves performance, but also reduces training time, achieving reductions between 60% and 90% compared to previous approaches shown in related works.

The contributions of this paper can be summarized as follows:

- 1) **Optimal Cycle Count Strategy:** This study employs a pairwise ranking model with Stable-Baselines3 DRL algorithms to provide efficient and stable test case prioritization. This indicates a systematic approach for assessing the influence of different cycle counts on DRL training to improve execution efficiency and prioritize accuracy. By determining the optimal cycle count for each DRL algorithm-dataset combination, the strategy reduces the required number of episodes compared with previous DRL-based TCP techniques.
- 2) **Adaptive Training Framework Using Meta-DRL:** This study introduces a novel Meta-Deep Reinforcement Learning (meta-DRL) framework that dynamically optimizes essential training hyperparameters, including learning rate, discount factor, cycle count, and number of episodes based on real-time performance feedback received during agent evaluation. This adaptability significantly improves the effectiveness of DRL agents across different testing scenarios. To the best of our knowledge, this paper is the first application of meta-DRL to prioritizing test cases in CI environments, indicating a significant achievement in

the integration of meta-learning with software testing automated processes.

- 3) **Comparative Analysis of DRL Algorithms across Different Datasets:** The effectiveness of various DRL algorithms, such as A2C, DQN, PPO, and TRPO, was thoroughly assessed using both basic and enriched datasets. This study provides valuable insights into the strengths and adaptability of the algorithms within CI contexts.

The rest of this paper is organized as follows: Section II provides a background study of the related research topic. Section III provides a review of related works, focusing on existing approaches for test case prioritization and the role of reinforcement learning in CI environments, and presents current developments in meta-DRL and its expanding function in supporting adaptive learning. Section IV details the proposed methodology, including the design of the DRL-based framework and selected algorithms (A2C, DQN, PPO, and TRPO). It also describes the experimental setup, including datasets, evaluation metrics, and implementation details. Section V presents the results and analysis, demonstrating the effectiveness of the proposed approach through comparative performance evaluation. Finally, Section VII concludes the paper by summarizing the key findings and discussing future research directions.

II. BACKGROUND STUDY

This section elaborates on DRL, meta-DRL definitions, and related terminology.

A. DEEP REINFORCEMENT LEARNING DEFINITION AND TERMINOLOGIES

DRL is a subfield of machine learning in which an agent interacts with the environment, observes the states within the environment, and completes actions to maximize the total reward received by the agent [11]. At each time step t , the agent receives a state from the environment as an input s_t , where $s_t \in S$, and S is the set of possible states [10]. Based on a given state, the agent selects and acts a_t from the action space within the environment. The agent selects an action using a policy that determines the behavior of the agent. Eventually, the environment responds by providing reward r_t to the agent and moving the agent to a new state until the agent reaches the terminal state, which determines the end of an episode [3]. An episode is defined as a sequence of interactions from the initial state to the final state. Both state and action spaces may be discrete or continuous based on the nature of the problem faced. Likewise, DRL methods can be classified into model-based DRL and model-free DRL [3], [10]. In this study, model-free DRL algorithms are employed because the results of the test cases at a specific CI cycle and their execution times are not known until they are executed. Additionally, all DRL algorithms that support a discrete action space were selected because of the implementation of a pairwise ranking

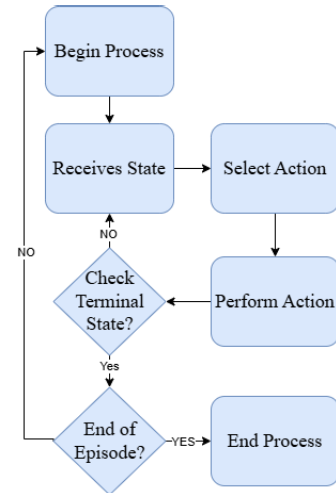


FIGURE 1. Deep reinforcement learning cycle.

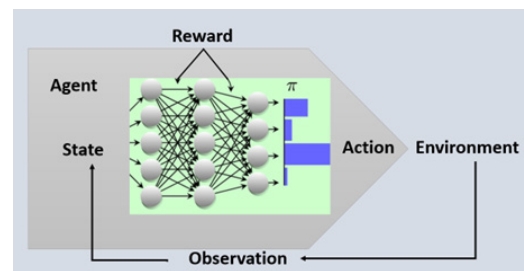


FIGURE 2. Deep reinforcement learning scheme.

model for test case prioritization. Figure 1 shows the DRL life cycle, which denotes the loop between various steps during an episode. Figure 2 shows the interaction between an agent and an environment in a reinforcement learning setup. In DRL, certain essential hyperparameters influence the learning dynamics and optimization effectiveness of the agent. The learning rate α determines the magnitude of the adjustments made to the parameters in response to new experiences. Higher learning rates can hinder the model from learning successfully, whereas lower learning rates might slow convergence. Moreover, owing to the irregularity of the objective function, the optimal learning rate may vary during the training process [12]. The discount factor γ , where $\gamma \in [0, 1]$, regulates the agent's assessment of future rewards; values approaching 1 promote long-term planning, whereas lower values lead the agent to prioritize current rewards [13]. Episode refers to the complete sequence of interaction starting from an initial state and ending in a terminal state, guided by actions, observations, and rewards, where the agent takes an action, receives a reward, and updates its policy (π), as shown in Figure 2. The cycle count denotes the frequency at which the agent undergoes retraining within a CI environment. Table 1 summarizes the hyperparameters and their roles. Moreover, iterations are used in training, where the cycle count refers to the number of previous cycles that have already been completed by the

TABLE 1. Summary of hyperparameters and their roles.

Hyperparameters	Meaning
Learning Rate (α)	Adjustments made to the parameters in response to new experiences.
Discount Factor (γ)	Trade-off parameter regulating current reward vs long-term return.
Episode (E)	Complete sequence of interaction.
Cycle Count (c)	Frequency at which the agent undergoes retraining.

agent and will be utilized in the next training iteration. During training, agents may require numerous episodes to acquire knowledge successfully. Training over episodes enables the agent to develop better policies by adapting to the accumulated rewards obtained across several actions [14].

1) STATE-OF-THE-ART DEEP REINFORCEMENT LEARNING ALGORITHMS

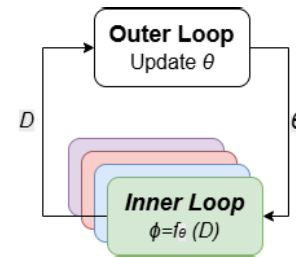
DRL has emerged as a powerful approach to solving complex decision-making problems by combining reinforcement learning techniques with deep neural networks. Among the various DRL algorithms, policy-gradient and value-based methods have gained significant attention for their ability to handle high-dimensional state and action spaces.

This study evaluates four DRL algorithms, each with distinct properties and capabilities, detailed as follows:

- **A2C:** Advantage Actor-Critic is a policy-gradient method based on synchronous implementation, an alternative variant of the asynchronous implementation of A3C, which computes the relative advantage of a particular action compared to other actions in the action space [15].
- **DQN:** Deep Q-Network is the DRL algorithm that combines the Q-network algorithm and deep neural networks to approximate the Q-Function using the neural network architecture [16].
- **PPO:** Proximal Policy Optimization is also a type of DRL policy-gradient method that maps actions to states by directly optimizing the policy function [17].
- **TRPO:** Trust Region Policy Optimization is another DRL policy-gradient method that directly optimizes the policy function, but with more constraints than PPO to prevent policies from deviating too far by keeping them in the trust regions [18].

B. META-DEEP REINFORCEMENT LEARNING DEFINITION

Meta-DRL considers a family of machine learning (ML) methods that learn reinforcement learning, which enhances standard DRL by shifting the focus from manually designing and tuning DRL algorithms to automatically learning algorithm components through experience. Instead of learning a policy for a particular task, meta-DRL seeks to learn a fundamental function (f) which is an adaptive DRL strategy that may produce effective policies across various challenges. This approach does not completely remove human participation but rather shifts it from manually developing the DRL algorithm to designing the meta-training

**FIGURE 3.** The interaction between the inner loop and outer loop in the meta-DRL.

environments and defining the learning parameters essential for data-driven algorithm enhancement. An outer and inner loop often characterizes the meta-DRL framework. The learning algorithm f is the outer loop, while the inner loop implements the learned f to enhance task-specific behavior [19]. The interaction between the inner loop and outer loop is illustrated in Figure 3.

The *outer loop* (meta-level) enhances collective knowledge across activities, represented by meta-parameters θ . The *inner loop* (task-level) rapidly adjusts to a particular task by utilizing minimal experience from that work. The inner loop is defined by the function f_θ , where θ represents the set of meta-parameters acquired from several tasks. This function accepts task-specific experience D , referred to as a meta-trajectory comprising several episodes, and generates a set of customized policy parameters ϕ , specifically designed for that task: $f_\theta(D) = \phi$, so f_θ learns how to update ϕ based on observed data and uses the task-specific experience data D to produce an adapted policy parameter ϕ tailored to that task [19].

III. RELATED WORK

The integration of reinforcement learning algorithms in test case prioritization has been an important area of study for many years [10], [20], [21]. Mindom et al. [3] utilized DRL algorithms to improve the prioritizing approach of test cases, making it adaptable to change in a CI environment. Utilizing simple and enriched datasets from prior CI cycles, the DRL agent can prioritize test cases to optimize fault detection at the beginning of the testing phase. In this study, A2C, PPO, DDPG, and DQN, derived from different frameworks (Stable-Baselines, Keras-rl, Tensorforce), were evaluated for their effectiveness in prioritizing test cases. The results indicate that when applied to the test case prioritization problem, the Stable-Baselines framework outperforms the Tensorforce framework in terms of accuracy.

To enhance CI practices, Bertolino et al. [22] introduces ten such algorithms and conducted an exhaustive comparative study, focusing on subjects drawn from the Apache Commons project. They also evaluated the impact of various factors, such as code attributes and the test process itself. This research included seven machine learning algorithms and three reinforcement learning algorithms that were assessed against each other, providing insights into their relative

performance. By applying these algorithms to subjects of the Apache Commons project, this study systematically evaluated their efficiency and effectiveness. This research not only considers the algorithms, but also explores the influence of diverse attributes. The code characteristics under test and the complexity of the testing process were examined to understand how they affect the performance of different algorithms. The outcomes of this study serve as valuable guidelines for testers seeking to optimize test case selection and prioritization techniques. By understanding how specific algorithmic approaches perform under varying conditions, testers can make informed decisions regarding which technique to adopt and how to fine-tune it for their specific context. Moreover, Bagherzadeh et al. [10] played a significant role in minimizing integration issues, accelerating development cycles, and speeding the release timelines. Nevertheless, CI's advantages come coupled with challenges in quality assurance practices, particularly in regression testing. They focus on addressing these challenges by using specific techniques tailored to the dynamic nature of CI and time constraints, such as A2C, ACER, ACKTR, DQN, and other state-of-the-art RL techniques.

Drawing inspiration from reinforcement learning succeed in dynamic and adaptive scenarios, such as game playing [23] and real-time bidding, this research extends its application to TCP in a CI environment. These techniques facilitate automated learning of a test case prioritization strategy that aims to approach optimality. The outcomes highlight a significant accuracy enhancement, with the newly proposed reinforcement learning solutions exhibiting prioritization strategies that are remarkably close to optimal. Challenges in adapting reinforcement learning to the unique environment of CI have been acknowledged and addressed through the exploration of tailored techniques. The study suggests that reinforcement learning can be effectively used to improve test case prioritization within CI practices, thereby improving the efficiency of regression testing.

In addition, Lima and Vergilio [24] proposed ranking-to-learn strategies that have emerged as particularly suitable for dynamic constraints of CI. These strategies utilize historical prioritization data guided by reward functions to learn the optimal prioritization patterns for specific code commits. In the context of contributing to advancements and shaping future research, they evaluated the effectiveness of ranking-to-learn approaches in comparison to deterministic approaches, which serve as ground truth. Specifically, the research investigates i) RETECS and ii) COLEMAN using huge datasets and various evaluation metrics. However, RETECS and COLEMAN demonstrate applicability in real-world scenarios and offer solutions that closely approximate the optimal solutions. RETECS aligns closely with optimality in 92% of cases, whereas COLEMAN [25] achieved the same result in 75% of the cases. The study also identified the specific limitations of these approaches. Notably, both approaches encounter challenges when dealing

with large test case sets, where faults are distributed across numerous test cases.

Finally, the RETECS approach, proposed by Spieker et al. [20], addresses the challenge of adaptive test case selection and prioritization in a CI environment. RETECS offers an automated system that utilizes reinforcement learning to assess attributes through analysis of duration, execution history, and failures. The empirical evidence derived from case studies conducted in industrial settings demonstrates the practicality and effectiveness of the technique in enhancing CI operations and regression testing. This study introduces "RETECS," an innovative technique designed to automate the process of learning the selection and prioritization of test cases within the context of CI. Numerous related studies have explored the application of machine learning and reinforcement learning for TCP; however, they typically employ predefined training configurations and manually optimized parameters, which lack adaptability in dynamic CI environments.

Specifically, prior research, such as that by Bagherzadeh et al. [10], required extensive training episodes (up to 200), resulting in increased computing expenses and limited immediate feedback during CI runs. Furthermore, none of the existing related studies incorporates a mechanism for the dynamic adjustment of learning parameters in relation to dataset complexity. These limitations underscore the necessity for a flexible, low-resource, DRL-based solution that learns and adapts efficiently in real-time.

In numerous practical applications, the capacity to adapt rapidly to new situations during deployment is crucial. Meta-DRL facilitates the acquisition of adaptation techniques across a spectrum of related tasks and offers a robust framework to meet this requirement. Figure 4 shows that meta-DRL exhibits significant efficacy across various domains, including traffic signal control, intelligent energy management, gaming, and multi-agent systems. This feature has been investigated in areas such as automated education and code evaluation, where learning agents must dynamically adapt to changing input conditions. Robotics has emerged as a significant application domain in which meta-DRL enables strong generalization across diverse task settings [19].

We first analyze important meta-learning techniques, such as Model-Agnostic Meta-Learning (MAML), which highlight meta-learning's capacity to enhance adaptation across various tasks. Finn et al. [26] introduced Model-Agnostic Meta-Learning (MAML), a robust meta-learning methodology designed to facilitate adaptation across diverse workloads. MAML does not include any learned parameters for meta-learning. It can be combined with any model representation appropriate for gradient-based training and any differentiable objective, including classification, regression, and reinforcement learning. However, extending the methodology to large datasets or more complicated jobs seems to be a significant research challenge. Furthermore, Finn et al. [27] later developed Follow-The-Meta-Leader (FTML) for

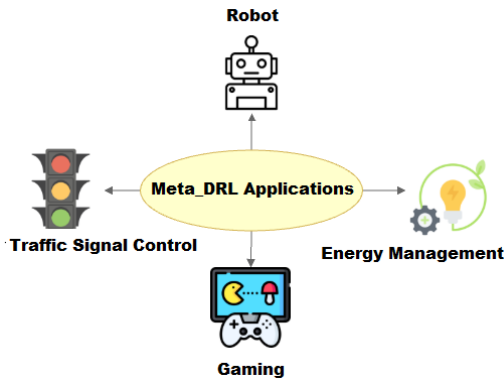


FIGURE 4. Meta-DRL applications.

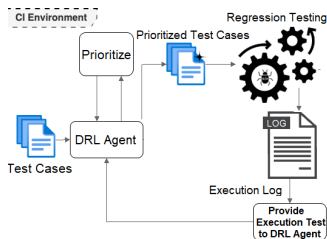


FIGURE 5. The integration of a DRL agent within the CI environment.

online meta-learning. FTML extends the MAML concept to non-stationary task sequences by organizing meta-learning within an online regret-minimization framework. The method inherits convexity under smoothness conditions, achieves an $O(\log T)$ regret bound against adaptive comparators, and provides a practical deep-learning implementation. Empirical results on vision benchmarks show improved forward transfer and data efficiency compared to online baselines. While FTML offers strong theoretical guarantees and demonstrates practical improvements, it still faces limitations such as computational and memory inefficiency and reliance on expensive meta-gradient updates. MAML and FTML guided the proposed design by emphasizing the significance of a bi-level, model-agnostic framework and the necessity for robust online adaptation in the context of dynamic task sequences. However, Table 2 presents a comparative analysis of existing reinforcement learning-based frameworks for test case prioritization, including the proposed meta-DRL.

To the best of our knowledge, this study is the first to apply meta-DRL to the problem of test case prioritization within CI environments. We introduce a novel framework that adaptively adjusts core learning parameters, such as the learning rate, discount factor, cycle count, and episode, based on real-time performance trends to enhance the agent's performance.

IV. METHODOLOGY AND EXPERIMENTS

This section explains the materials and methods used in the experiments, which ultimately led to the desired proposed model. Figure 5 shows the integration of a DRL agent

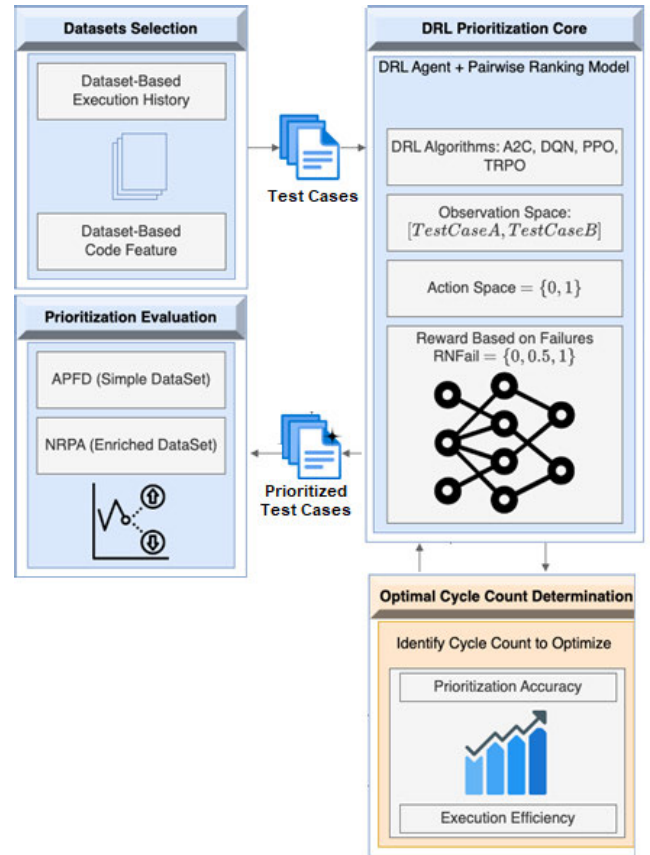


FIGURE 6. Overview of the optimal cycle count strategy within a DRL-Based TCP framework for CI environment.

within the CI environment for prioritizing regression test cases. The evolution of this methodology includes multiple enhancement phases. Initially, the focus was on determining the optimal cycle count for each combination of DRL algorithm and dataset, as illustrated in Figure 6. Motivated by the positive findings of this phase, the methodology was further enhanced by implementing an adaptive training framework based on meta-DRL, as depicted in Figure 7. An essential component of this study is the formulation of well-defined research questions, as outlined in Section IV-A. To answer the research questions, three separate experiments were performed; each experiment was designed to assess the impact of optimal cycle count strategy and meta-DRL on the performance of various DRL algorithms across different datasets in the context of test case prioritization within CI environments.

A. RESEARCH QUESTIONS

Based on the current discussion, we can confidently pose the research questions. Three research questions were formulated, and the results covering all research questions were provided at the end of the work.

- **RQ1:** How can the proposed technique be applied in a CI environment to optimize test case prioritization, and

TABLE 2. Comparison of reinforcement learning frameworks for test case prioritization.

Framework	Adaptability	Training Cost	Real-Time Applicability
RETECTS [20]	Adapts only within fixed parameters and a single environment.	Achieves effective performance after approximately 60 CI cycles.	Constrained, as it is tailored mainly to history-based test case prioritization datasets.
TP_RL [10]	Works with both small and large-scale datasets, but only with static configurations.	Requires up to 200 training episodes.	Applicable, though dependent on substantial computational resources and higher execution cost.
Meta-DRL (Proposed)	Observes performance during training and dynamically adjusts hyperparameters.	Requires a dynamically adjusted, limited number of training episodes.	Strongly adaptive, enabling robust prioritization across different CI datasets in real time.

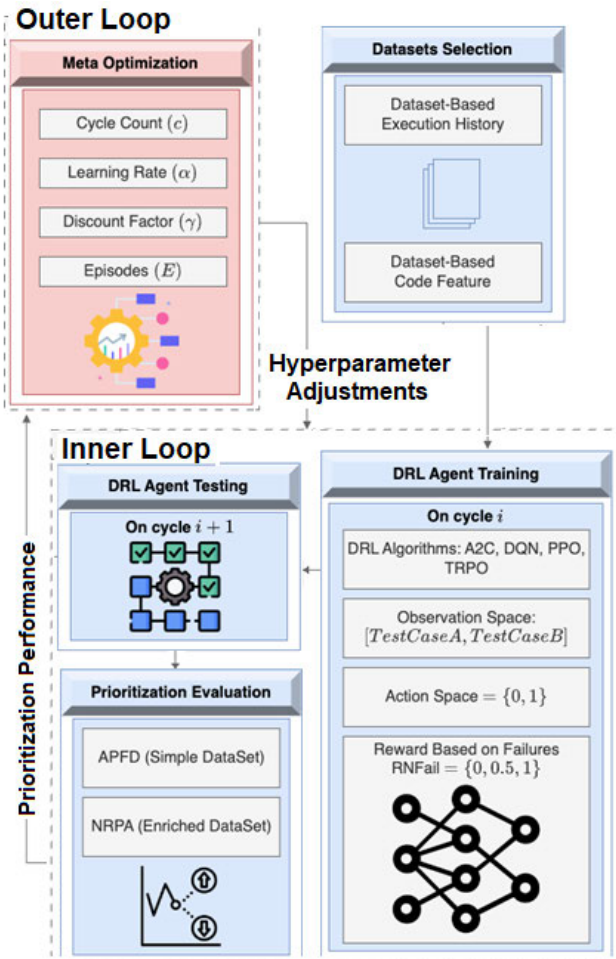


FIGURE 7. The proposed meta-DRL methodology for adaptive test case prioritization in CI environments.

what is the optimal cycle count for each combination of algorithms and datasets to achieve the best results?

- **RQ2:** How does the use of the optimal cycle count affect the performance of the selected DRL algorithms when applied to both simple and enriched datasets?
 - **RQ2.1:** Which combination of the DRL algorithm and dataset yields the best prioritization accuracy?
 - **RQ2.2:** Which combination of the DRL algorithm and dataset offers the best performance in terms of training time and prediction time?
- **RQ3:** How effective is meta-DRL in dynamically adapting training hyperparameters to enhance the

performance of a DRL-based TCP framework across CI environments?

- **RQ3.1:** To what extent can meta-DRL dynamically adapt training hyperparameters to improve test case prioritization accuracy while simultaneously reducing training time within CI environments?
- **RQ3.2:** How does the meta-DRL framework perform relative to baseline approaches, and to what extent do the Friedman and Wilcoxon signed rank tests statistically validate the comparative performance of algorithms?

B. DATASETS

The DRL algorithms were implemented on eight datasets, which were classified into two categories: simple and enriched. The simple datasets mainly consisted of the execution history derived from industrial datasets obtained from ABB Robotics Norway. These datasets were utilized to test complex industrial robots, namely the Paint Control Dataset and IOF/ROL Dataset [20]. The simple datasets contain features such as age, pass/fail verdicts, and execution time. The enriched dataset included code-based features such as lines of code and cyclomatic complexity derived from source code. We adopted code-based features from the literature [22] that are extracted using light-weight and incremental static analysis and repository mining techniques. The datasets were derived from six Java subjects found in the open-source Apache Commons project. This category contains six datasets as follows: Codec, Compress, Imaging, Io, Lang, and Math Dataset [22]. Table 3 lists the characteristics of each dataset. The execution logs consist of up to 438 CI cycles, each including at least six test cases. The logs column indicates the total number of test case execution logs, varying from 2,207 to 32,118. Enriched datasets have a low number of failed cycles and a low failure rate. Simple datasets, on the other hand, have a high number of failed cycles and a high failure rate. The last column shows the average computation time of enriched features per cycle [10]. Given that datasets exhibit varying failure rates, section IV-C explains and justifies the use of dataset-specific evaluation metrics.

C. EVALUATION METRICS

We employed two evaluative metrics, the Average Percentage of Faults Detected (APFD) and the Normalized Rank Percentile Average (NRPA).

TABLE 3. Datasets and characteristics.

Dataset	Type	CI Cycles	Logs	Fail Rate	Failed Cycles	Avg. Calculation Time of Enriched Features per Cycle [22]
IOFROL	Simple	209	32,118	28.66	203	NA
Paint-Control	Simple	332	25,568	19.36	252	NA
CODEC	Enriched	178	2,207	0	0	1.78
IMAG	Enriched	147	4,482	0.04	2	5.60
IO	Enriched	176	4,985	0.06	3	2.88
COMP	Enriched	438	10,335	0.06	7	3.64
LANG	Enriched	301	10,884	0.01	2	5.58
MATH	Enriched	55	3,822	0.01	7	9.46

1) AVERAGE PERCENTAGE OF FAULTS DETECTED (APFD)

The APFD metric is designed to evaluate the effectiveness of test case prioritization techniques [28], [29] and calculates the weighted average of the percentage of faults detected during test case execution. This metric was utilized in the baseline study by Bagherzadeh et al. [10]. The APFD is defined as follows, adopted from [3], as shown in Equation 1:

$$APFD(s_e) = 1 - \frac{\sum_{t \in s_e} idx(s_e, t) \cdot t.v}{|s_e| \cdot m} + \frac{1}{2 \cdot |s_e|} \quad (1)$$

where:

- s_e : Prioritized sequence of test cases.
- $t \in s_e$: Test case t in sequence s_e .
- $idx(s_e, t)$: The index for test case t in sequence s_e .
- $t.v$: Execution verdict for test case t .
- $|s_e|$: Total number of test cases in sequence s_e .
- m : Total number of faults.

2) NORMALIZED RANK PERCENTILE AVERAGE (NRPA)

The NRPA metric is designed to determine the accuracy of ranking the test cases, as shown in Equations 2 and 3 [30]. However, it is not suitable in the presence of failures. NRPA is appropriate when context does not matter and measures how close a predicted ranking of test cases is to the optimal ranking. It has also been used in related research papers [10]. The NRPA is defined as follows, as adopted from [3]:

$$RPA = \frac{\sum_{m \in s} (\sum_{i=idx(s,m)}^k |s| - idx(s_o, m) + 1)}{k^2(k+1)/2} \quad (2)$$

$$NRPA = \frac{RPA(s_e)}{RPA(s_o)} \quad (3)$$

where:

- s_e : Sequence generated by ranking algorithm.
- s_o : The optimal sequence is.
- k : The total number of test cases.
- $m \in s$: Each item m in ranked sequence s .
- $idx(s, m)$: The index of item m in the sequence s .
- $idx(s_o, m)$: The index of item m in the optimal sequence s_o .

While NRPA is effective for general ranking evaluation, it can be misleading in the presence of failures, as it treats all test cases equally regardless of the execution verdict. Bagherzadeh et al. [10] highlighted this limitation, reporting that NRPA values often contradict APFD values in datasets,

and therefore recommended APFD as a complementary metric to measure how well a ranking reveals fault early. To ensure both fairness and comparability with prior work, particularly the baseline study by Bagherzadeh et al. [10], we employed both APFD and NRPA [22]. Taken together, these metrics provide a more balanced and rigorous evaluation of prioritization effectiveness across the proposed DRL configurations.

D. COMPARISON BASELINE

We adopted the study of Bagherzadeh et al. [10] as a baseline. It represents one of the most recent and effective RL-based TCP frameworks for CI. Their results indicated that RL-based TCP consistently outperformed three established baselines, with detailed experimental findings made publicly available.

E. REWARD FUNCTION

The performance of the selected action, which involves prioritized test cases, was assessed through the definition of a reward function, as shown in Equation 4 [21]. In the current study, the RNFail reward function was employed, which was initially presented by Spieker et al. [20], with some modifications. This reward function is designated as a reward based on failures. However, RNFail, as shown in Equation 4 adopted from [24], provides a formula for t'_c , which represents a test case derived from a prioritized test suite T'_c in cycle c , where the failure reward function is determined by the failures connected to the test case as follows:

$$RNFail(t'_c) = \begin{cases} 1 & \text{if } t'_c \text{ failed} \\ 0 & \text{otherwise} \end{cases} \quad (4)$$

Bagherzadeh et al. [10] employed an improved version of the RNFail (Equation 4) reward function to adapt it to the pairwise ranking model, including the observation and action spaces. The implementation details are outlined below.

- **Observation space:** An agent's observation consists of a pair of test cases.
- **Action space:** The values within the action space are limited to 0 or 1.
- **Reward function:** The agent prefers the failed test case in the pair. Therefore, it obtains the maximum reward (1); otherwise, it receives no reward (0). If the test cases in the pair return identical verdicts, the agent

is awarded (0.5) when the test case with the shorter execution time is prioritized; otherwise, it is 0. However, the reward (0.5) emphasizes that when the significance of fault detection is equivalent, minimizing execution time becomes a further goal, while prioritizing early fault detection remains the primary goal of this reward function.

F. PAIRWISE RANKING MODEL

The proposed framework integrated a pairwise ranking model, commonly used in DRL for test case prioritization in CI environments, due to its considerable advantages over listwise and pointwise ranking models. The pairwise ranking model is a machine learning strategy that ranks items by comparing pairs based on their relative scores [31]. It is mostly utilized to establish rankings in search engine retrieval and recommendation processes. Therefore, pairwise ranking emphasizes the comparison of two test cases simultaneously, thus streamlining the ranking process into a sequence of binary classification tasks, which enhances processing efficiency [32]. Further pairwise ranking shows stable behavior in handling inconsistent data [33], particularly in CI environments that frequently update test cases and codebases. Pairwise models are the most widely used among learning-to-rank methodologies [33], [34], as they generate optimal results when integrated with DRL algorithms, as demonstrated in a relevant research study [10]. In the proposed implementation, the pairwise ranking model was invoked at the start of each DRL episode. The initial observation involved a chosen pair of test cases, with the agent selecting either 0 or 1; 0 indicated that the first test case had a higher priority and should be executed first. This experiment employed the merge sort algorithm [10]. Throughout this process, the environment updated observations in accordance with the sorting concepts of the merge sort algorithm, allowing the agent to progressively enhance prioritized decisions.

G. OPTIMAL CYCLE COUNT STRATEGY

To have an efficient agent learning within the DRL-based framework for prioritizing test cases, determining the optimal cycle count is initially deemed necessary. Instead of using a fixed value, the cycle count is modified and evaluated across multiple configurations until an optimal balance is achieved for each combination of the dataset and the DRL algorithm. This balance aims to maximize prioritization performance while minimizing the training time required. A manual search strategy was used to determine optimal cycle counts by systematically testing different values. The cycle count that yielded the highest performance was identified for each dataset. The resulting quality of prioritization is assessed as follows. We used APFD for simple datasets and NRPA for enriched datasets. As illustrated in Figure 6, the proposed framework incorporates a manual search phase to determine the optimal cycle count within

the DRL prioritization process. This process forms the foundation for further adaptation in our meta-DRL extension, where dynamic hyperparameter updates are built on these empirically grounded cycle count values. Optimal cycle counts were important in the meta-DRL technique, serving as starting points for each dataset. By utilizing these experimentally derived values instead of random hypotheses, the meta-DRL framework effectively minimized unnecessary exploration and focused on improving cycle counts during training, eventually enhancing the model's overall performance.

H. ADAPTIVE TRAINING FRAMEWORK USING META-DRL

Previously, DRL algorithms such as DQN required manual tuning of hyperparameters, which involved experimenting with various combinations of critical parameters such as cycle count, learning rate, episodes, and discount factor, and assessing their effect on performance through multiple training iterations. This technique is time-consuming and resource-intensive and often leads to suboptimal results, especially in the context of test case prioritization in CI environments, where training time increases as CI cycles increase. To address these limitations, this study presents an adaptive training framework based on meta-DRL. This methodology divides the DRL agent into two loops: an inner loop and an outer loop.

The inner loop employs a DRL algorithm (e.g., PPO) that learns to prioritize test cases by maximizing the reward functions linked to metrics such as APFD and NRPA. The outer loop is typically designated as a meta-optimization loop. It learns the ability to adapt by optimizing hyperparameters that enhance the task-specific performance of the inner loop across multiple tasks. The performance trends of the inner agent were managed, and the training configuration was dynamically modified. The outer loop dynamically adjusts hyperparameters such as the learning rate, discount factor, number of episodes, and cycle count in response to the observed variations in prioritizing performance over time. Algorithm 1 shows the details of the meta-DRL approach for adaptive test case prioritization in CI environments. Hence, the outer loop increases or decreases the values of the hyperparameters based on the performance trends at time during training to obtain optimal results. This adaptive feedback process mitigates the possibility of underfitting or overfitting commonly associated with static configurations, such as training with insufficient or excessive episodes, or employing an unsuitable learning rate or cycle count. The meta-DRL framework allows the agent to “learn how to learn” by substituting manual adjustment with an adaptive, performance-oriented optimization approach. This enables the framework to achieve more dependable and efficient learning results in the test case prioritization process within CI environments. The proposed methodology is illustrated in Figure 7, which presents a meta-DRL-enhanced framework for test case prioritization in CI environments.

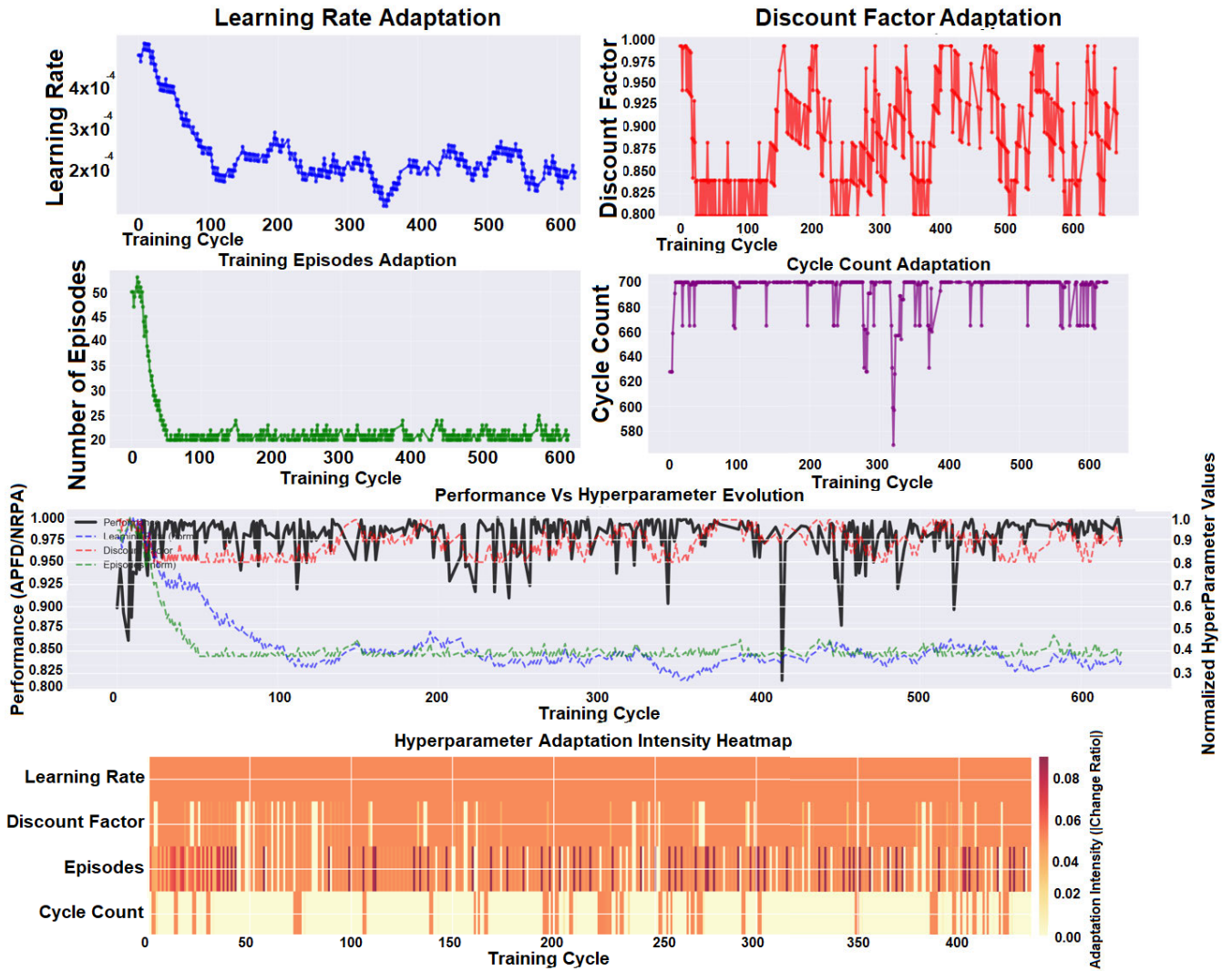


FIGURE 8. Meta-DRL learning-to-learn analysis.

I. ALGORITHM AND COMPLEXITY

The algorithm and its complexity in terms of both space and time are discussed in this section.

Algorithm 1 aims to dynamically adjust the parameters of the DRL algorithm based on performance feedback. The performance metric (perf) in meta-DRL is explicitly defined as APFD or NRPA, depending on the type of dataset. Furthermore, the main loop updates key hyperparameters, such as learning rate, discount factor, episodes, and cycle count, depending on performance deltas, as shown in Figure 8.

1) ANALYSIS

- Let n denote the number of training cycles, defined as:

$$n = \text{end_cycle} - \text{start_cycle}$$

- The algorithm iterates through each cycle using a for loop (line 4):

for each i from start_cycle to $\text{end_cycle}-1$

- Within each iteration, if `meta_DRL_enabled` and `prev_perf` are not `None`, the algorithm executes a series of parameter updates (lines 5–31):

- Computes the performance difference: $\Delta\text{perf} = \text{current_perf} - \text{prev_perf}$ (line 6)
- Adjusts:
 - Cycle Count (c) using scalar arithmetic (lines 7–11)
 - Learning Rate (α) using scalar arithmetic (lines 13–17)
 - Discount Factor (γ) using scalar arithmetic (lines 19–23)
 - Number of Episodes (E) using scalar arithmetic (lines 25–29)
- All these operations involve simple conditionals and scalar updates, which run in constant time.

2) TIME COMPLEXITY

- Each iteration of the loop performs a constant number of operations: comparisons, additions, multiplications, and calls to `min` or `max` functions.

Algorithm 1 Adaptive Meta-DRL for Test Case Prioritization

```

1: Input: algo  $\leftarrow$  DRL algorithm (A2C, DQN, PPO,
   TRPO), mode  $\leftarrow$  Pairwise strategy, dataset_type  $\leftarrow$  CI
   dataset (simple, enriched)
2: Initialize: c (cycle count),  $\alpha$  (learning rate),  $\gamma$  (discount
   factor), E (episodes),  $\mu$  (meta adjustment factor)
3: prev_perf  $\leftarrow$  None
4: for each cycle index i from start_cycle to end_cycle - 1
   do
5:   if meta_DRL_enabled and prev_perf  $\neq$  None then
6:      $\Delta_{\text{perf}} \leftarrow \text{current\_perf} - \text{prev\_perf}$ 
7:     Adjust Cycle Count (c)
8:     if  $|\Delta_{\text{perf}}| < \epsilon$  then
9:        $c \leftarrow \max(c_{\min}, c \times (1 - \mu))$ 
10:    else if  $\Delta_{\text{perf}} < 0$  then
11:       $c \leftarrow \min(c_{\max}, c \times (1 + \mu))$ 
12:    end if
13:    Adjust Learning Rate ( $\alpha$ )
14:    if  $\Delta_{\text{perf}} < 0$  then
15:       $\alpha \leftarrow \alpha \times (1 + \mu)$ 
16:    else
17:       $\alpha \leftarrow \alpha \times (1 - \mu)$ 
18:    end if
19:    Adjust Discount Factor ( $\gamma$ )
20:    if  $\Delta_{\text{perf}} < 0$  then
21:       $\gamma \leftarrow \min(0.99, \gamma \times (1 + \mu))$ 
22:    else
23:       $\gamma \leftarrow \max(0.8, \gamma \times (1 - \mu))$ 
24:    end if
25:    Adjust Episodes (E)
26:    if  $\Delta_{\text{perf}} < 0$  then
27:       $E \leftarrow \min(E_{\max}, E \times (1 + \mu))$ 
28:    else
29:       $E \leftarrow \max(E_{\min}, E \times (1 - \mu))$ 
30:    end if
31:    Print updated values of c,  $\alpha$ ,  $\gamma$ , and E
32:  end if
33:  prev_perf  $\leftarrow$  current_perf
34: end for

```

- There are no nested loops or operations that scale with the size of the input dataset within the main loop.
- Therefore, the time complexity per iteration is $\mathcal{O}(1)$.
- Over *n* iterations, the total time complexity is: $\mathcal{O}(n)$

3) SPACE COMPLEXITY

- The algorithm maintains a fixed number of scalar variables: *c*, α , γ , *E*, μ , Δ_{perf} , *prev_perf*, and *current_perf*.
- No lists, matrices, or data structures that grow with the input sizes are used in this portion of the algorithm.
- Thus, the space complexity is: $\mathcal{O}(1)$

Time Complexity: $\mathcal{O}(n)$, where *n* is the number of training cycles. **Space Complexity:** $\mathcal{O}(1)$, as the algorithm uses only a constant amount of memory.

This analysis assumes that *current_perf* is computed externally or retrieved in constant time. If *current_perf* involves additional computation (e.g., model evaluation over test cases), that complexity must be separately accounted for.

V. RESULTS AND DISCUSSION

This section presents the results of the experiments. For reliable and comprehensive results, the experiments and their respective outcomes were aligned with the RQs. The following sections present the results related to the respective RQs.

A. RESULTS FOR RQ1

The proposed technique was employed to enhance test case prioritization in a CI environment by utilizing the Gymnasium open-source library [36] to simulate the CI environment through execution logs by applying the DRL algorithms implemented by Stable-Baselines3 [37]. The gymnasium is an upgraded derivation of Gym [38], which incorporates numerous enhancements and API modifications to facilitate its ongoing application in open-source deep reinforcement learning research [36]. Stable-Baselines3 DRL algorithms were applied, specifically A2C, DQN, PPO, and TRPO, to optimize test case prioritization in a CI environment. Each algorithm was applied independently on all datasets, ensuring a direct and fair performance comparison. In alignment with the study by Bagherzadeh et al. [10], the default cycle count was set at 100. Therefore, we conducted an initial experiment using 100-cycle counts to establish a baseline for comparison. In contrast to the previous related study [10] that mainly utilized Stable-Baselines2, we adopted Stable-Baselines3, an updated and improved version of the original framework. The primary objective of this work was to reduce the number of episodes while maintaining high performance in test case prioritization. Reducing the number of episodes in the DRL enhances the efficiency and minimizes the computational overhead. This approach is particularly crucial when data collection is costly or time-consuming, as shown in test case prioritization problems or other real-world scenarios. For example, the research study by Lee et al. [35] proposed an Episodic Backward Update (EBU) method to improve efficiency by propagating rewards backward through episodes and reducing the need for multiple episodes. Table 4 shows the averages and standard deviations of all experiments for the deep reinforcement learning algorithms across eight datasets using 100-cycle counts, which lasted for 50 episodes. The experiments were conducted on a system equipped with a Core i7 processor and NVIDIA RTX 3060 GPU. The use of this configuration ensured sufficient computational capacity for training and evaluation. The results presented in Table 4 are not encouraging, suggesting room for improvement. Therefore, numerous experiments were conducted on four different DRL algorithms across eight datasets with varying cycle counts to determine the optimal cycle counts for each DRL algorithm-dataset combination, as shown in Table 5.

TABLE 4. The average performance of different configurations of the DRL algorithm and dataset in terms of APFD and NRPA using 100-cycle counts.

Algorithm	IOFROL (APFD)	Paint. (APFD)	CODEC (NRPA)	IMAG (NRPA)	IO (NRPA)	COMP (NRPA)	LANG (NRPA)	MATH (NRPA)
A2C	0.55 ± 0.16	0.58 ± 0.27	0.82 ± 0.16	0.96 ± 0.05	0.98 ± 0.03	0.97 ± 0.03	0.94 ± 0.04	0.94 ± 0.05
DQN	0.51 ± 0.13	0.55 ± 0.24	0.89 ± 0.13	0.95 ± 0.05	0.98 ± 0.03	0.96 ± 0.04	0.93 ± 0.05	0.94 ± 0.07
PPO	0.56 ± 0.13	0.58 ± 0.22	0.90 ± 0.11	0.94 ± 0.07	0.97 ± 0.03	0.95 ± 0.04	0.94 ± 0.04	0.94 ± 0.05
TRPO	0.55 ± 0.16	0.58 ± 0.25	0.89 ± 0.06	0.93 ± 0.05	0.98 ± 0.04	0.96 ± 0.04	0.94 ± 0.04	0.96 ± 0.03

TABLE 5. Optimal cycle counts for each DRL algorithm-dataset combination.

Dataset	Optimal Cycle Count
IOFROL	320
Paint-Control	352
CODEC	614
IMAG	377
IO	386
COMP	628
LANG	522
MATH	112

B. RESULTS FOR RQ2

Table 6 presents the averages and standard deviations of APFD for simple datasets (IOFROL, Paint-Control) and NRPA for enriched datasets (CODEC, IMAG, IO, COMP, LANG, MATH) across the four DRL algorithms: A2C, DQN, PPO, and TRPO over 50 episodes. Thus, Table 6 indicates a significant improvement in the results achieved through optimal cycle counts compared to the results presented in Table 4 utilizing 100-cycle counts. The accuracy achieved through optimal cycle counts is comparable in some cases or slightly better than that in the related work [10] by Bagherzadeh et al., despite using only 50 episodes compared to their 200 episodes across all experiments.

C. RESULTS FOR RQ2.1

After beginning with the analysis of a simple dataset, the results for IOFROL closely corresponded to the results of a previous study [10]. The best algorithm in this dataset was TRPO with an APFD score of 0.56 ± 0.14 . Figure 9 (a) shows the performance of different configurations of the DRL algorithms with the IOFROL dataset across 209 cycles. In comparison to the related study [10], the Paint-Control results show an improvement in accuracy, especially for DQN, showing improved prioritization with the optimal cycle count. Additionally, A2C performs better on this dataset than the other DRL algorithms, with an APFD of 0.71 ± 0.24 , suggesting that it can efficiently identify the faults early. Figure 9 (b) shows the performance of different configurations of DRL algorithms with the Paint-Control dataset across 332 cycles. The enriched datasets CODEC, IO, and COMP yielded results comparable to the baseline study [10] when optimal cycle counts were applied, utilizing only 50 episodes. The findings for CODEC indicated that A2C, PPO, and TRPO showed comparable results. Figure 9 (c) shows the performance of different configurations of the DRL algorithms with the CODEC dataset across 178 cycles. All algorithms demonstrated a strong performance on the

IO dataset, with A2C and DQN achieving notable NRPA scores. Moreover, the A2C algorithm ranked among the top performers in the COMP dataset. Figure 9 (d) and 9 (e) illustrate the performance of various configurations of the DRL algorithms using the IO and COMP datasets over 176 and 438 cycles, respectively. However, IMAG, LANG, and MATH demonstrated slight improvements in certain DRL algorithms compared to related work [10]. A2C and DQN demonstrated better results in terms of NRPA scores for the IMAG dataset. This indicates that both DRL algorithms effectively prioritize the test cases within this dataset. In the LANG dataset, A2C demonstrated a slight edge over the others and outperformed A2C in a related work [10]. In the MATH dataset, TRPO demonstrated some improvements over other DRL algorithms, achieving an NRPA of 0.96. A2C and PPO followed closely, with NRPA values of 0.95. Figure 9 (f), 9 (g), and 9 (h) illustrate the performance of various DRL algorithm configurations using the IMAG, MATH, and LANG datasets. This comparison highlights the importance of tuning the cycle counts to enhance the performance across various datasets and DRL algorithms, as shown in Figure 9.

D. RESULTS FOR RQ2.2

For the training time, the time that the DRL agents spent training their policy was calculated, which included 50 episodes. By analyzing the provided Table 7, which contains training times in minutes for various combinations of DRL algorithms and datasets, utilizing only 50 episodes. The goal here is to minimize the training time while maintaining high performance in test case prioritization. It is obvious that all the training time results, except one configuration (A2C-IOFROL), are significantly better than the results provided by Bagherzadeh et al. in the relative research paper [10]. For a simple IOFROL dataset, optimal performance was attained with TRPO, requiring a training time of 477.0. In the Paint-Control dataset, PPO provided the most rapid training time at 388.7. Furthermore, for the enriched datasets IMAG, IO, LANG, and MATH, TRPO consistently demonstrated superior performance, with minimal training time. Additionally, for the CODEC dataset, A2C was most rapidly processed at 43.1, closely succeeded by TRPO at 46.3. For the COMP dataset, DQN demonstrates superior performance with a training duration of 201.3. Figure 10 shows the training time of DRL algorithms for all datasets using an optimal cycle count.

Table 8 presents the averages and standard deviations of prediction time for all experiments. The time required

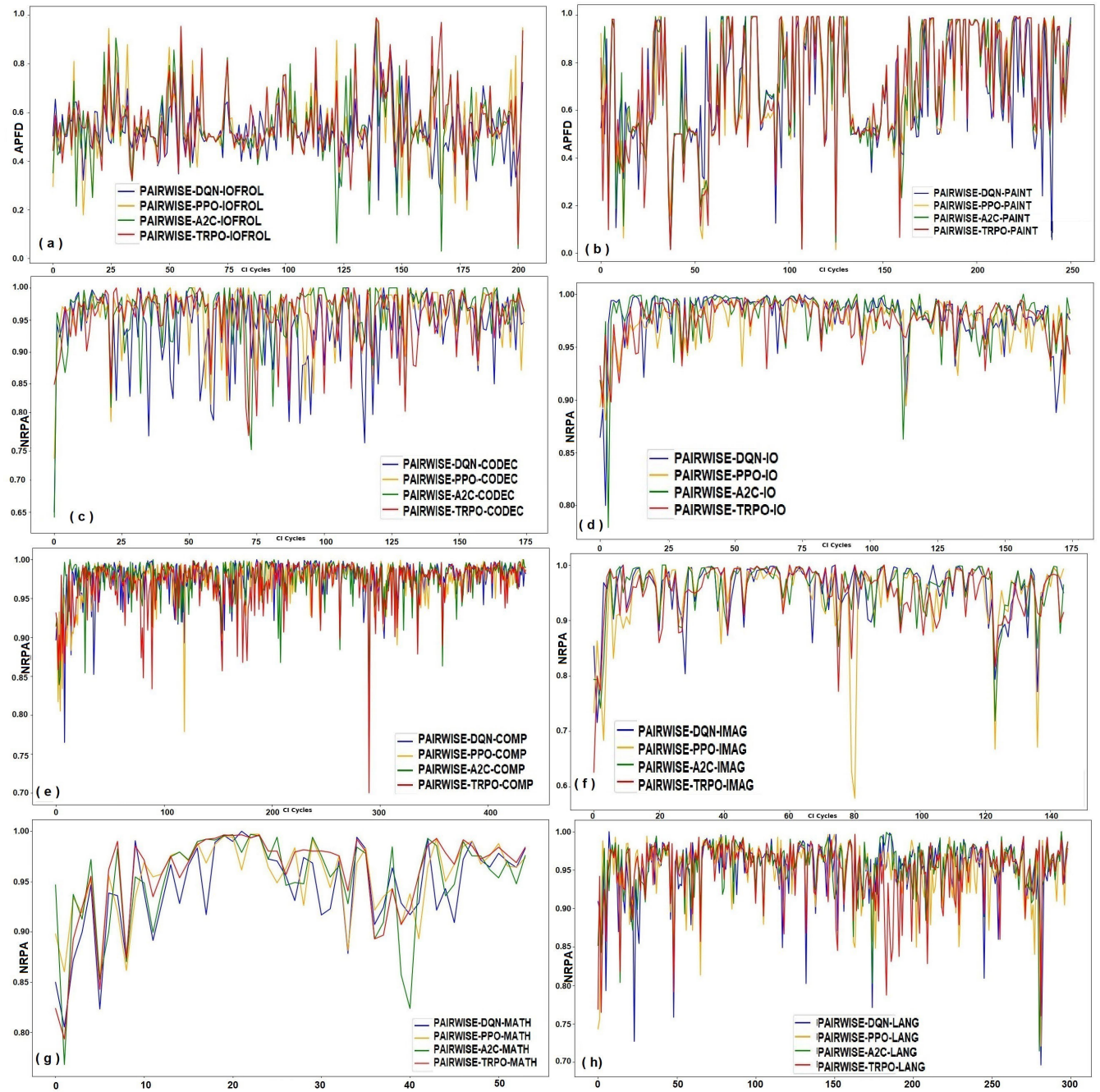


FIGURE 9. The average performance of different configurations of DRL algorithm and datasets in terms of APFD and NRPA using optimal cycle counts. (a) APFD for all DRL algorithms and the IOFROL dataset. (b) APFD for all DRL algorithms and Paint-Control dataset. (c) NRPA for all DRL algorithms and CODEC dataset. (d) NRPA for all DRL algorithms and IO dataset. (e) NRPA for all DRL algorithms and COMP datasets. (f) NRPA for all DRL algorithms and IMAG datasets. (g) NRPA for all DRL algorithms and MATH dataset. (h) NRPA for all DRL algorithms and LANG dataset.

TABLE 6. The average performance of different configurations of DRL algorithm and datasets in terms of APFD and NRPA using optimal cycle counts.

Algorithm	IOFROL (APFD)	Paint. (APFD)	CODEC (NRPA)	IMAG (NRPA)	IO (NRPA)	COMP (NRPA)	LANG (NRPA)	MATH (NRPA)
A2C	0.54 ± 0.15	0.71 ± 0.24	0.96 ± 0.05	0.96 ± 0.05	0.98 ± 0.03	0.98 ± 0.03	0.96 ± 0.04	0.95 ± 0.05
DQN	0.53 ± 0.11	0.68 ± 0.23	0.94 ± 0.06	0.96 ± 0.05	0.98 ± 0.03	0.97 ± 0.03	0.95 ± 0.04	0.94 ± 0.05
PPO	0.55 ± 0.15	0.70 ± 0.24	0.96 ± 0.04	0.95 ± 0.07	0.97 ± 0.02	0.97 ± 0.03	0.95 ± 0.04	0.95 ± 0.04
TRPO	0.56 ± 0.14	0.70 ± 0.24	0.96 ± 0.05	0.95 ± 0.05	0.98 ± 0.02	0.97 ± 0.03	0.95 ± 0.04	0.96 ± 0.05

for DRL agents to prioritize several test cases was calculated. All results provided in Table 8 outperform

those presented by Bagherzadeh et al. in the related study [10]. According to Table 8, the DQN consistently

TABLE 7. Total training time (Minutes) of DRL algorithms across simple and enriched datasets using optimal cycle count.

Algorithm	IOFROL	Paint.	CODEC	IMAG	IO	COMP	LANG	MATH
A2C	1476.7	396.6	43.1	76.1	103.2	211.7	240.2	73.2
DQN	728.2	465.6	49.0	68.3	87.1	201.3	199.9	78.6
PPO	613.4	388.7	59.8	90.9	95.2	275.0	203.9	75.0
TRPO	477.0	733.1	46.3	61.2	72.5	283.9	179.5	55.4

TABLE 8. The average prediction time (Seconds) of DRL algorithms across simple and enriched datasets using optimal cycle count.

Algorithm	IOFROL	Paint.	CODEC	IMAG	IO	COMP	LANG	MATH
A2C	1.06±1.73	0.45±0.32	0.07±0.04	0.15±0.11	0.18±0.13	0.11±0.07	0.22±0.20	0.44±0.67
DQN	0.57±0.88	0.31±0.21	0.07±0.08	0.10±0.06	0.10±0.06	0.07±0.04	0.12±0.09	0.28±0.42
PPO	0.96±1.55	0.45±0.33	0.07±0.05	0.17±0.12	0.14±0.10	0.11±0.08	0.17±0.14	0.45±0.69
TRPO	0.95±1.52	0.42±0.35	0.06±0.04	0.14±0.10	0.14±0.09	0.16±0.23	0.18±0.15	0.46±0.74

demonstrated superior performance over most of the datasets.

E. RESULTS FOR RQ3

In this study, the optimal cycle counts for retraining the DRL agent through empirical experimentation for each dataset were initially identified. Although this static configuration led to improved prioritization performance, it remains highly dataset-specific and lacks adaptability, limiting its applicability in real-world CI environments where codebases and testing conditions evolve dynamically over time. To mitigate this limitation, we propose an adaptive training framework that automates the tuning process via the integration of meta-DRL. This framework improves the process by adjusting cycle counts dynamically based on feedback data such as performance (e.g., APFD or NRPA variations) and codebase instability, optimizing essential training hyperparameters, including the learning rate and discount factor, in real time throughout the agent's training process. This adaptive technique allows the framework to extend more successfully across various CI environments, minimizing manual intervention and retaining high-priority accuracy with improved execution efficiency.

F. RESULTS FOR RQ3.1

Table 9 demonstrates that meta-DRL consistently attains high prioritization accuracy across eight datasets. All four DRL algorithms (A2C, DQN, PPO, and TRPO) exhibited commendable performance under the meta-DRL framework. PPO and TRPO attained APFD (≥ 0.70) on Paint and robust NRPA scores (≥ 0.94) across all enriched datasets. DQN, although slightly lower in performance on simple datasets (e.g., 0.53 on IOFROL), attained elevated NRPA scores (e.g., 0.95–0.98) across most enriched datasets. A2C, whereas competitive had slightly more variance in the simpler datasets. The results demonstrate that meta-DRL efficiently enables each DRL algorithm to adjust to various dataset complexities and CI situations, resulting in robust adaptation. Figure 11 illustrates the performance of various DRL algorithm configurations across different datasets using meta-DRL.

In terms of the total training time, it was observed that meta-DRL reduced the need for unnecessary retraining, which led to the efficient utilization of computational resources while preserving high-quality prioritization. Table 10 indicates TRPO has a faster training convergence rate, proving it appropriate for scenarios where efficiency is a critical issue. For the IOFROL dataset, DQN and A2C show the highest training times at 461.7 and 441.3, respectively, whereas PPO and TRPO require only 176.9 and 141.2, amounting to a 60–70% reduction in training time. In the Paint-Control dataset, PPO and TRPO reduce training time by over 60% compared to A2C and DQN. DQN exhibits efficient performance in several enriched datasets. The training time observed in the conducted experiments was reduced by approximately 60% to 90% compared to the training time reported in the baseline [10]. To regulate convergence and eliminate redundant computation, early stopping was employed during training. Specifically, if no improvement was detected for 50 consecutive checks, early stopping was triggered and the most recent best-performing model was preserved for evaluation. This strategy enabled adaptive control of training duration and reduced variance across training runs while maintaining optimal performance. Figure 12 shows the training time of DRL algorithms across all datasets utilizing meta-DRL.

G. RESULTS FOR RQ3.2

The proposed meta-DRL methodology achieves prioritizing accuracy that is mostly comparable to the 200-episodes baseline achieved by Bagherzadeh et al. [10]. In certain instances, the baselines achieve marginally higher accuracy, whereas in others, meta-DRL provides better outcomes. As shown in Figure 13, the NRPA comparison on the LANG dataset across the four DRL algorithms highlights that performance varies depending on the algorithm, with no single approach consistently outperforming the other. Furthermore, the proposed framework significantly reduces training time, achieving a decrease of around 60%–90% compared to the baseline. This enhancement arises from the adaptive properties of the meta-DRL, facilitating rapid modification of essential hyperparameters. Figure 14 illustrates

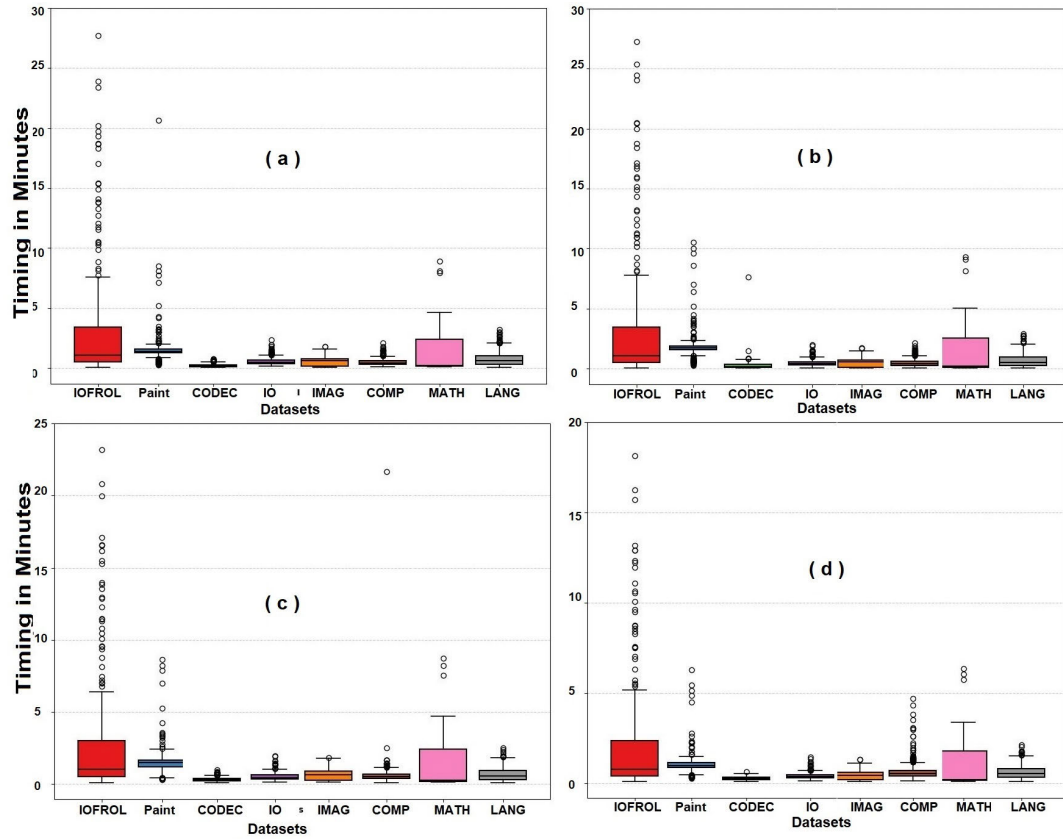


FIGURE 10. Training time of DRL algorithms across simple and enriched datasets using optimal cycle count. (a) Training time of A2C for all datasets. (b) Training Time of DQN for all datasets. (c) Training time of PPO for all datasets. (d) Training time of TRPO for all datasets.

TABLE 9. The average performance of different configurations of DRL algorithm and datasets in terms of APFD and NRPA using meta-DRL.

Algorithm	IOFROL (APFD)	Paint. (APFD)	CODEC (NRPA)	IMAG (NRPA)	IO (NRPA)	COMP (NRPA)	LANG (NRPA)	MATH (NRPA)
A2C	0.56 ± 0.16	0.70 ± 0.24	0.96 ± 0.04	0.96 ± 0.06	0.98 ± 0.02	0.98 ± 0.03	0.96 ± 0.04	0.96 ± 0.04
DQN	0.53 ± 0.12	0.69 ± 0.24	0.95 ± 0.06	0.96 ± 0.05	0.98 ± 0.02	0.97 ± 0.03	0.95 ± 0.04	0.95 ± 0.05
PPO	0.56 ± 0.14	0.70 ± 0.25	0.95 ± 0.05	0.95 ± 0.05	0.97 ± 0.02	0.97 ± 0.03	0.94 ± 0.04	0.95 ± 0.05
TRPO	0.56 ± 0.15	0.71 ± 0.23	0.96 ± 0.04	0.95 ± 0.05	0.97 ± 0.03	0.97 ± 0.03	0.95 ± 0.04	0.96 ± 0.04

TABLE 10. Total training time (Minutes) of DRL algorithms across simple and enriched datasets using meta-DRL.

Algorithm	IOFROL	Paint.	CODEC	IMAG	IO	COMP	LANG	MATH
A2C	441.3	286.1	33.9	48.8	55.5	138.4	119.5	56.1
DQN	461.7	169.2	21.8	44.4	52.0	126.6	133.7	61.7
PPO	176.9	107.9	57.7	50.9	59.7	168.0	165.6	52.5
TRPO	141.2	91.1	43.1	39.1	53.4	140.8	123.7	41.9

the comparison between the training times of the proposed meta-DRL framework and the baseline, utilizing four DRL algorithms over eight datasets.

Table 11 presents the average prediction time (in seconds) for the four DRL algorithms evaluated across eight datasets. Overall, the results indicate that TRPO and DQN consistently achieve favorable prediction times across both simple and enriched datasets under the meta-DRL framework. TRPO and PPO achieved significantly lower prediction times than A2C and DQN on the IOFROL datasets. Meanwhile, DQN shows

competitive low prediction times, especially in enriched datasets such as CODEC ($0.06 \pm 0.03s$), IO ($0.11 \pm 0.07s$), and COMP ($0.09 \pm 0.05s$).

However, we employed the Friedman test to detect differences across multiple test attempts. The test statistic follows a chi-square (X^2) distribution. This method is widely applied for comparing multiple related samples. As shown in Figure 15, LANG, COMP, IO, and IMAG datasets provide strong evidence of significant differences, where higher bars indicate more statistically significant results. CODEC and

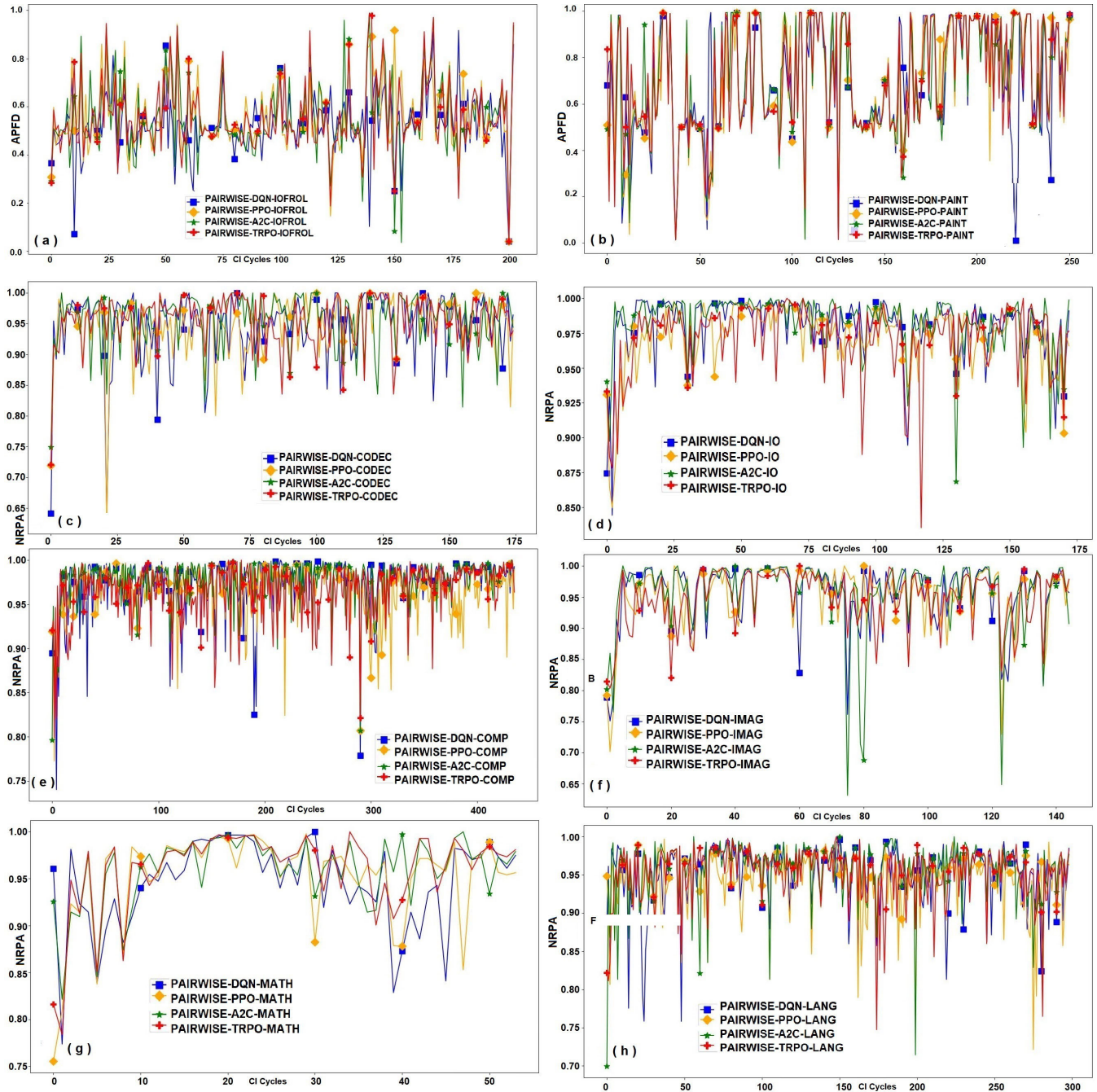


FIGURE 11. The average performance of different configurations of DRL algorithm and datasets in terms of APFD and NRPA using meta-DRL. (a) APFD for all DRL algorithms and the IOFROL dataset. (b) APFD for all DRL algorithms and Paint-Control dataset. (c) NRPA for all DRL algorithms and CODEC dataset. (d) NRPA for all DRL algorithms and IO dataset. (e) NRPA for all DRL algorithms and COMP datasets. (f) NRPA for all DRL algorithms and IMAG datasets. (g) NRPA for all DRL algorithms and MATH dataset. (h) NRPA for all DRL algorithms and LANG dataset.

TABLE 11. The average prediction time (Seconds) of DRL algorithms across simple and enriched datasets using meta-DRL.

Algorithm	IOFROL	Paint.	CODEC	IMAG	IO	COMP	LANG	MATH
A2C	1.09 ± 1.64	0.59 ± 0.47	0.09 ± 0.06	0.17 ± 0.12	0.16 ± 0.10	0.13 ± 0.09	0.20 ± 0.16	0.48 ± 0.70
DQN	0.63 ± 1.00	0.19 ± 0.11	0.06 ± 0.03	0.11 ± 0.07	0.11 ± 0.07	0.09 ± 0.05	0.14 ± 0.11	0.30 ± 0.44
PPO	0.49 ± 0.76	0.24 ± 0.15	0.09 ± 0.05	0.16 ± 0.10	0.15 ± 0.10	0.12 ± 0.08	0.23 ± 0.19	0.46 ± 0.68
TRPO	0.49 ± 0.75	0.25 ± 0.18	0.07 ± 0.04	0.16 ± 0.10	0.15 ± 0.10	0.12 ± 0.07	0.21 ± 0.17	0.46 ± 0.71

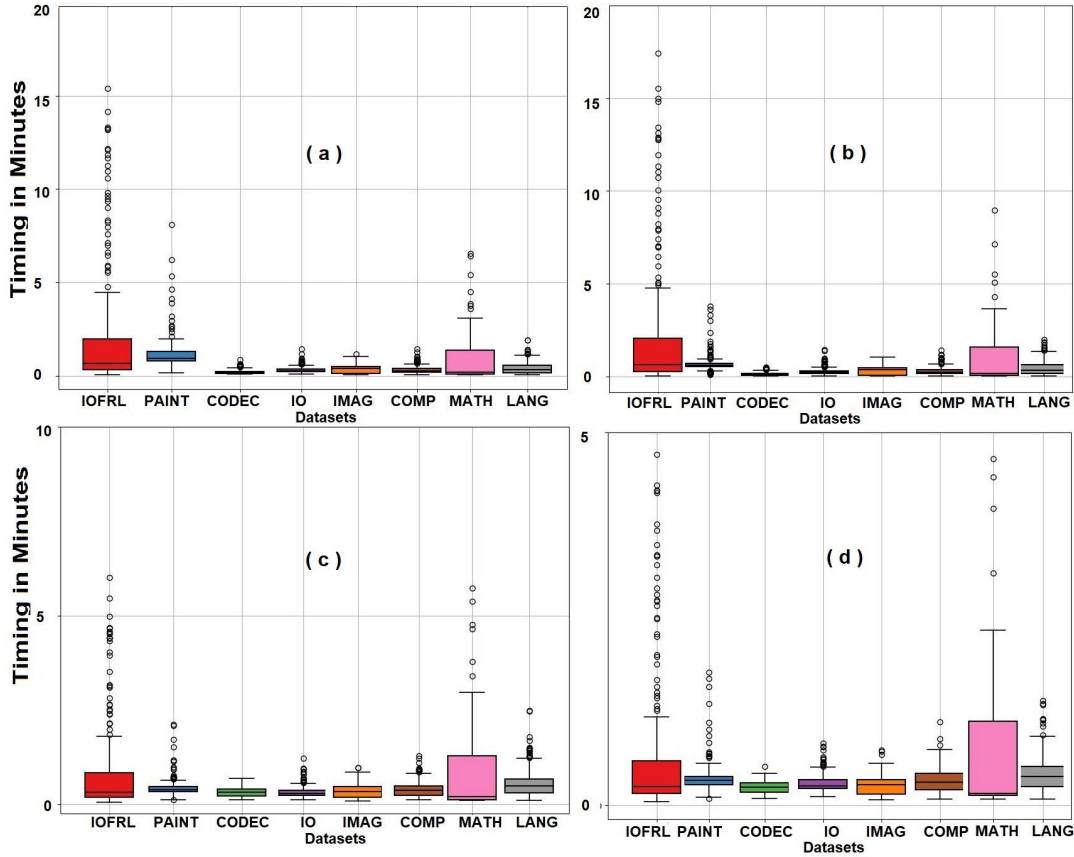


FIGURE 12. Training time of DRL algorithms across simple and enriched datasets using meta-DRL. (a) Training time of A2C for all datasets. (b) Training Time of DQN for all datasets. (c) Training time of PPO for all datasets. (d) Training time of TRPO for all datasets.

MATH suggest moderate evidence, whereas IOFROL and Paint-Control show no evidence of meaningful differences across groups. The key findings are as follows:

- 1) Highly significant results ($p < 1e - 12$): The LANG dataset exhibits the strongest significance ($\chi^2 = 122.54$, $p < 1e - 12$), followed by COMP ($\chi^2 = 105.64$, $p < 1e - 12$), IO ($\chi^2 = 99.21$, $p < 1e - 12$), and IMAG ($\chi^2 = 75.49$, $p < 1e - 12$).
- 2) Moderately significant results: The CODEC dataset shows moderate significance ($\chi^2 = 36.43$, $p = 6.08e - 08$), followed by MATH ($\chi^2 = 13.50$, $p \approx 0.00$).
- 3) Non-significant results: No statistical significance was observed for IOFROL ($\chi^2 = 2.18$, $p = 0.54$) or Paint-Control ($\chi^2 = 3.51$, $p = 0.32$).

Figure 16 presents statistical test results for the CODEC dataset, which involves four different DRL algorithms. The analysis uses two complementary statistical tests to understand if there are significant differences between four different algorithms. The Friedman test yielded $X^2 = 36.43$ with a p-value of $6.078e-08$. Since the p-value is far below the 0.05 threshold, the results indicate strong statistical significance. This result confirms that there are meaningful

differences in performance among the four algorithms, with at least some algorithms perform differently from the others. Specific comparisons reveal the following outcomes: DQN vs. PPO ($p = 0.82$) and DQN vs. TRPO ($p = 0.54$) show no significant differences, while DQN vs. A2C ($p = 0.01$) indicates a significant difference. PPO vs. A2C ($p < 1e-06$) and A2C vs. TRPO ($p = 1.39e-06$) demonstrate highly significant differences, whereas PPO vs. TRPO ($p = 1.00$) indicates no measurable difference. Key insights from these comparisons highlight that A2C behaves significantly differently from all other algorithms. In contrast, PPO and TRPO exhibit nearly identical performance ($p = 1.00$), while DQN shows intermediate behavior, aligning more closely with PPO and TRPO.

H. DISCUSSION

This study definitively shows that DRL algorithms have significant potential for addressing in TCP challenges in CI environments. The application of DRL algorithms, such as A2C, DQN, PPO, and TRPO, when integrated with a pairwise ranking model, yields increased fault detection rates and greater prioritization efficiency across many datasets. The initial experiments conducted with a fixed 100-cycle count provided a useful starting point. However, they

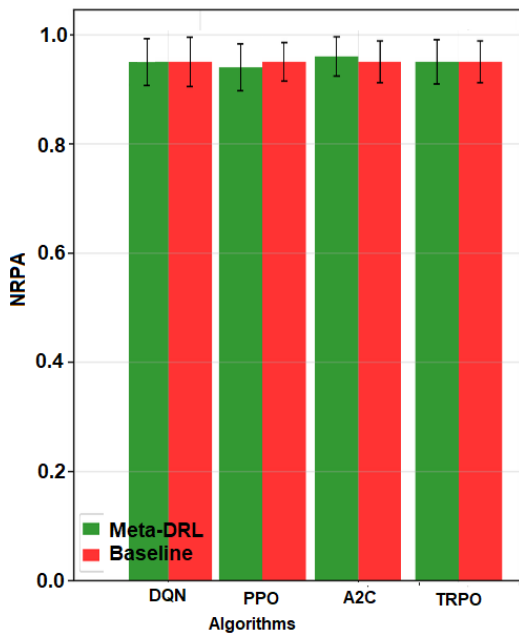


FIGURE 13. Comparison of the LANG dataset performance between the proposed meta-DRL framework and the baseline across four DRL algorithms (DQN, PPO, A2C, and TRPO).

revealed limitations in prioritization accuracy and required a substantial number of training episodes, resulting in significant computational overhead when applied to large-scale datasets. This means that there is still potential that has not been fully realized. Later, cycle counts were changed based on the characteristics of each dataset and algorithm. This led to a big improvement in performance. The only way to solve these problems and reach the goals was to find the optimal cycle count. The DRL algorithms efficiently optimized the balance between the time needed for training and the accuracy of prioritizing by changing the frequency of retraining based on the complexity and characteristics of each dataset. The enriched datasets CODEC and COMP, characterized by higher levels of complexity, experienced advantages from increased cycle counts compared to the simpler structures of IOFROL, requiring fewer retraining cycles. The finding of the optimal cycle counts significantly contributed to improving the efficiency of the DRL-based test case prioritization. However, the process for determining the optimal cycle count typically requires human adjustment for each dataset-algorithm combination, primarily depending on empirical experience and prior domain knowledge. To mitigate this constraint and enhance adaptability, this approach is extended by introducing meta-DRL, allowing the system to independently modify cycle counts and other critical training parameters based on performance feedback. Meta-DRL enhances the optimal cycle count technique by extending it into a self-adaptive learning-to-learn mechanism suitable for dynamic CI contexts.

The improved values of APFD and NRPA across both simple and enriched datasets demonstrate the adaptability

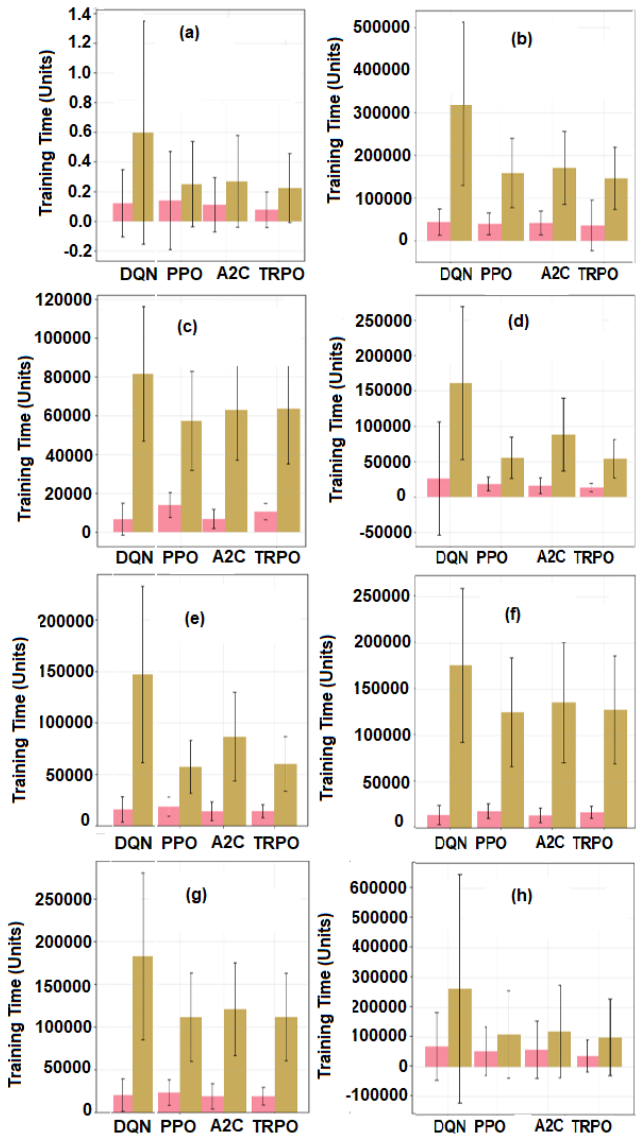


FIGURE 14. Comparison of training times for meta-DRL (pink) and the baseline (gold) across four DRL algorithms with confidence intervals: (a) IOFROL, (b) Paint-Control, (c) CODEC, (d) IMAG, (e) IO, (f) COMP, (g) LANG, and (h) MATH datasets.

of the proposed framework. Within the optimal cycle count and meta-DRL experiments, the TRPO algorithm consistently showed strong APFD performance on simple datasets. TRPO is a policy-gradient method that directly improves the policy function. The trust-region optimization approach keeps policy updates constant and stops unexpected adjustments, which is especially helpful when fault detection accuracy (APFD) is critical [18]. On the other hand, PPO frequently had short training times on dataset-based execution history. The A2C method provided the most accurate and fastest training time for both the optimal cycle count and meta-DRL framework while working with enriched datasets. Actor-critic algorithms naturally combine the most effective parts of actor-only and critic-only

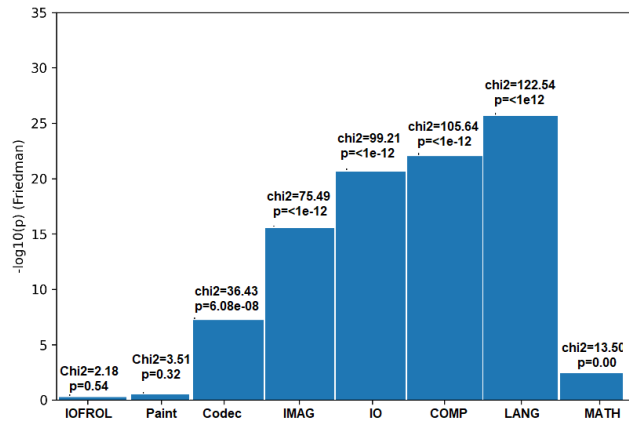


FIGURE 15. Friedman test results across eight datasets, where the y-axis represents $-\log_{10}(p)$ values.

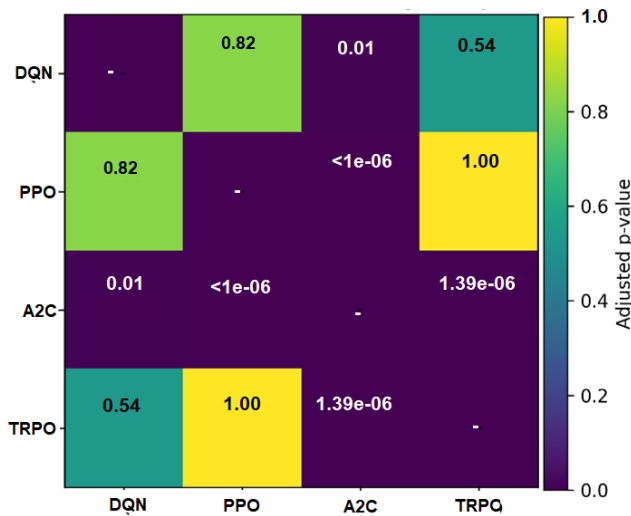


FIGURE 16. Wilcoxon test for the CODEC dataset illustrating the comparisons between DRL algorithms. Dark purple indicates strong significance, teal reflects moderate significance, light green denotes no significant difference, and yellow represents identical performance.

approaches. The critic estimates the value function, and the actor changes the policy distribution in the way that the critic suggests [15]. This integration allows actor-critic methods to find an adequate balance between accuracy, training time, and prediction performance [9]. Furthermore, DQN had average prediction speeds that were competitive, which made it a good choice for situations when quick decisions about what to do next are needed. The importance of meta-DRL goes beyond just better performance numbers. Meta-DRL has more benefits than the optimal cycle count strategy, making it more adaptable, scalable, and useful in CI environments. Meta-DRL dynamically alters hyperparameters, which lets the framework work with different types of CI datasets without needing to tune parameters for each. Meta-DRL significantly reduces retraining overhead by minimizing unnecessary episodes, even when accuracy improvements are relatively limited. This shortens overall training time and

lowers energy consumption, which is a critical consideration in large-scale industrial CI environments. Moreover, meta-DRL can eliminate the significant trial-and-error parameter tuning generally necessary when implementing DRL in a new CI context. The framework also makes it easier to scale because it can be used with both execution history-based datasets (APFD) and code feature-based datasets (NRPA) with minimal manual intervention. When using DRL for test case prioritization in a CI context, the choice between a fixed 100-cycle count, optimal cycle count, and meta-DRL can have a significant impact on performance. A fixed cycle count guarantees consistency across multiple experiments because each execution of test case prioritization is conducted for only 100 cycles, allowing comparisons across various datasets and algorithms. Assigning a fixed number of cycles also simplifies the process of establishing and training the model. However, applying an optimal cycle count enables the DRL model to be sufficiently trained to obtain the most effective prioritization method. The results reveal that adjusting the cycle counts optimizes the time utilized for training and computing resources, as shown in Table 7. On the other hand, adding more episodes makes the time needed for training significantly longer and uses more resources, which means that resources are not used efficiently across all datasets. As previously discussed, meta-DRL dynamically adjusts hyperparameters during the training process. This enables the framework to operate with multiple types of CI datasets without requiring explicit parameter adjustments for each dataset. Overall, utilizing a standardized cycle count facilitates more consistent comparisons among research. When datasets are limited, it is straightforward and beneficial to identify and utilize the optimal cycle count for each. This strategy becomes increasingly ineffective as the volume and diversity of datasets expand. In such cases, an adaptive framework such as meta-DRL is superior due to its ability to adjust according to the characteristics of each dataset. This adaptability reduces the effort of manual configuration while enhancing resource allocation and increasing the overall efficiency of CI environments.

VI. THREATS TO VALIDITY

When assessing the threats to validity linked to using DRL for Test Case Prioritization in CI environments, different threats to validity must be considered as follows:

A. INTERNAL VALIDITY

The effectiveness of DRL algorithms may significantly rely on the characteristics of the datasets employed in the experiments. For example, low failure rates, like that of the enriched dataset, are common to the datasets frequently used for assessing test selection and prioritization methodologies [39]. If the training datasets lack representativeness of actual CI contexts, the outcomes may not accurately represent the actual constraints of the DRL models. In the context of CI, the application of fault injection techniques is not feasible, as it would introduce arbitrary defects into the system.

B. EXTERNAL VALIDITY

Because of the high and low failure rates of the simple and enriched data sets, respectively, the results of our research may not be as generalizable to other CI environments with varied configurations and test case distributions. However, we just use these data sets to conduct a fair comparison with the related work [10], and we do not make any general claim on the efficiency of our model.

C. CONSTRUCT VALIDITY

The evaluation metrics employed in this study (APFD and NRPA) may not address all performance factors that are essential in the CI context. Other essential issues like resource usage, scalability, and fault tolerance may not have been thoroughly evaluated in this experiment as we explained the evaluation metrics in detail in section IV-C (Evaluation Metrics).

VII. CONCLUSION AND FUTURE WORK

This study has effectively illustrated a novel DRL-based TCP framework that integrates a pairwise ranking model with state-of-the-art DRL algorithms (A2C, DQN, PPO, and TRPO), employing an optimal cycle count strategy. An empirical evaluation showed that establishing the optimal cycle count markedly improves execution efficiency and prioritization accuracy. Motivated by these promising results, the methodology was further enhanced by implementing an adaptive training framework based meta-DRL. This framework allows the system to independently modify essential training parameters according to real-time performance feedback. We performed a comprehensive assessment across eight different datasets by combining four DRL algorithms, resulting in 32 various dataset–DRL algorithm combinations. The experiments indicate that the proposed meta-DRL framework provides great prioritization accuracy, as shown by the higher APFD and NRPA metrics in both the simple and enriched CI datasets. Meta-DRL is important since it not only enhances performance, but it also reduces training time by 60% to 90% compared to previous approaches shown in related works. However, this study mainly tests the proposed framework with benchmark datasets, which might not show the complete complexity of real-world CI environments. Additionally, the reward functions used are limited to APFD and NRPA, which means that other key metrics are not being investigated. We used publicly accessible datasets from previous studies to make comparisons, but these datasets are still limited because they typically have either very low or very high failure rates.

Finally, future work will focus on developing a dynamic, context-aware reward function specifically designed for the complexity of CI environments and advancing the DRL-based TCP framework by incorporating real-time CI data and adaptive learning techniques. Moreover, we aim to evaluate the effectiveness of DRL models on more complex datasets, so the evaluation criteria will be expanded beyond

APFD and NRPA to encompass other indicators, including Mean Time to Detect (MTTD) and cost-aware APFD. These extensions will provide a more profound understanding of the practical impact of meta-DRL in the CI context and strengthen the comparative analysis presented in this study.

REFERENCES

- [1] K. Zhang, Y. Zhang, L. Zhang, H. Gao, R. Yan, and J. Yan, "Neuron activation frequency based test case prioritization," in *Proc. Int. Symp. Theor. Aspects Softw. Eng. (TASE)*, Dec. 2020, pp. 81–88.
- [2] V. Tomar, M. Bansal, and P. Singh, "Regression testing approaches, tools, and applications in various environments," in *Proc. 4th Int. Conf. Artif. Intell. Speech Technol. (AIST)*, Dec. 2022, pp. 1–6.
- [3] P. S. N. Mindom, A. Nikanjam, and F. Khomh, "A comparison of reinforcement learning frameworks for software testing tasks," *Empirical Softw. Eng.*, vol. 28, no. 5, p. 111, Sep. 2023.
- [4] M. Waqar, Imran, M. A. Zaman, M. Muzammal, and J. Kim, "Test suite prioritization based on optimization approach using reinforcement learning," *Appl. Sci.*, vol. 12, no. 13, p. 6772, Jul. 2022.
- [5] E. Knauss, M. Staron, W. Meding, O. Söder, A. Nilsson, and M. Castell, "Supporting continuous integration by code-churn based test selection," in *Proc. IEEE/ACM 2nd Int. Workshop Rapid Continuous Softw. Eng.*, May 2015, pp. 19–25.
- [6] S. U. R. Khan, S. P. Lee, R. M. Parizi, and M. Elahi, "A code coverage-based test suite reduction and prioritization framework," in *Proc. 4th World Congr. Inf. Commun. Technol. (WICT)*, Dec. 2014, pp. 229–234.
- [7] R. Huang, W. Zong, J. Chen, D. Towey, Y. Zhou, and D. Chen, "Prioritizing interaction test suites using repeated base choice coverage," in *Proc. IEEE 40th Annu. Comput. Softw. Appl. Conf. (COMPSAC)*, vol. 1, Jun. 2016, pp. 174–184.
- [8] M. Khatibsyarhini, M. A. Isa, D. N. A. Jawawi, and R. Tumeng, "Test case prioritization approaches in regression testing: A systematic literature review," *Inf. Softw. Technol.*, vol. 93, pp. 74–93, Jan. 2018.
- [9] A. Sharif, D. Marijan, and M. Liaen, "DeepOrder: Deep learning for test case prioritization in continuous integration testing," in *Proc. IEEE Int. Conf. Softw. Maintenance Evol. (ICSME)*, Sep. 2021, pp. 525–534.
- [10] M. Bagherzadeh, N. Kahani, and L. Briand, "Reinforcement learning for test case prioritization," *IEEE Trans. Softw. Eng.*, vol. 48, no. 8, pp. 2836–2856, Aug. 2022.
- [11] F. Agostinelli, G. Hocquet, S. Singh, and P. Baldi, "From reinforcement learning to deep reinforcement learning: An overview," in *Proc. Int. Conf. Commemorating 40th Anniversary Emmanuil Braverman's Decease*, 2018, pp. 298–328.
- [12] H. Don acio, A. Barrier, L. F. South, and F. Forbes, "Dynamic learning rate for deep reinforcement learning: A bandit approach," 2024, *arXiv:2410.12598*.
- [13] M. Kim, J.-S. Kim, M.-S. Choi, and J.-H. Park, "Adaptive discount factor for deep reinforcement learning in continuing tasks with uncertainty," *Sensors*, vol. 22, no. 19, p. 7266, Sep. 2022.
- [14] R. S. Sutton, "Open theoretical questions in reinforcement learning," in *Proc. Eur. Conf. Comput. Learn. Theory*, 1999, pp. 11–17.
- [15] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Harley, T. Lillicrap, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *Proc. 33rd Int. Conf. Mach. Learn. ICML*, Feb. 2016, pp. 2850–2869.
- [16] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing Atari with deep reinforcement learning," 2013, *arXiv:1312.5602*.
- [17] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [18] J. Schulman, S. Levine, P. Abbeel, M. I. Jordan, and P. Moritz, "Trust region policy optimization," in *Proc. 32nd Int. Conf. Mach. Learn. ICML*, Jordan, 2015, pp. 1889–1897.
- [19] J. Beck, R. Vuorio, E. Zheran Liu, Z. Xiong, L. Zintgraf, C. Finn, and S. Whiteson, "A tutorial on meta-reinforcement learning," 2023, *arXiv:2301.08028*.
- [20] H. Spieker, A. Gotlieb, D. Marijan, and M. Mossige, "Reinforcement learning for automatic test case prioritization and selection in continuous integration," in *Proc. 26th ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2017, pp. 12–22.

- [21] H. Mirzaei and M. R. Keyvanpour, "Reinforcement learning reward function for test case prioritization in continuous integration," in *Proc. 9th Iranian Joint Congr. Fuzzy Intell. Syst. (CFIS)*, Mar. 2022, pp. 1–6.
- [22] A. Bertolino, A. Guerriero, B. Miranda, R. Pietrantuono, and S. Russo, "Learning-to-rank vs ranking-to-learn: Strategies for regression testing in continuous integration," in *Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng. (ICSE)*, Oct. 2020, pp. 1–12.
- [23] Y. Zheng, X. Xie, T. Su, L. Ma, J. Hao, Z. Meng, Y. Liu, R. Shen, Y. Chen, and C. Fan, "Wuji: Automatic online combat game testing using evolutionary deep reinforcement learning," in *Proc. 34th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Nov. 2019, pp. 772–784.
- [24] J. A. D. P. Lima and S. R. Vergilio, "An evaluation of ranking-to-learn approaches for test case prioritization in continuous integration," *J. Softw. Eng. Res. Develop.*, vol. 11, no. 1, pp. 1–4, Feb. 2023.
- [25] J. A. P. Lima and S. R. Vergilio, "A multi-armed bandit approach for test case prioritization in continuous integration environments," *IEEE Trans. Softw. Eng.*, vol. 48, no. 2, pp. 453–465, Feb. 2022.
- [26] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 1126–1135.
- [27] C. Finn, A. Rajeswaran, S. M. Kakade, and S. Levine, "Online meta-learning," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 1920–1930.
- [28] M. A. Mor, "Evaluate the effectiveness of test suite prioritization techniques using APFD metric," *IOSR J. Comput. Eng.*, vol. 16, no. 4, pp. 47–51, 2014.
- [29] Z. Wang, C. Fang, L. Chen, and Z. Zhang, "A revisit of metrics for test case prioritization problems," *Int. J. Softw. Eng. Knowl. Eng.*, vol. 30, no. 8, pp. 1139–1167, Aug. 2020.
- [30] Y. Zhao, D. Hao, and L. Zhang, "Revisiting machine learning based test case prioritization for continuous integration," in *Proc. IEEE Int. Conf. Softw. Maintenance Evol. (ICSME)*, Oct. 2023, pp. 232–244.
- [31] J. Fürnkranz and E. Hüllermeier, "Pairwise preference learning and ranking," in *Proc. Eur. Conf. Mach. Learn.*, 2003, pp. 145–156.
- [32] V. Melnikov, E. Hüllermeier, D. Kaimann, B. Frick, and P. Gupta, "Pairwise versus pointwise ranking: A case study," *Schedae Informaticae*, vol. 2016, pp. 73–83, Jan. 2017.
- [33] C. J. Burges, "From RankNet to LambdaRank to LambdaMART: An overview," *Learning*, vol. 11, nos. 23–581, p. 81, 2010.
- [34] X. Zhao, L. Zhang, Z. Ding, L. Xia, J. Tang, and D. Yin, "Recommendations with negative feedback via pairwise deep reinforcement learning," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2018, pp. 1040–1048.
- [35] S. Y. Lee, S. Choi, and S.-Y. Chung, "Sample-efficient deep reinforcement learning via episodic backward update," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 2110–2119.
- [36] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, K. G. Arjun, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, H. Tan, and O. G. Younis, "Gymnasium: A standard interface for reinforcement learning environments," 2024, *arXiv:2407.17032*.
- [37] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, Dec. 2021. [Online]. Available: <http://jmlr.org/papers/v22/20-1364.html>
- [38] H. Hori. (2016). *Reinforcement Learning Using OpenAI Gym*. Accessed: Feb. 24, 2023. [Online]. Available: <https://gym.openai.com/>
- [39] H. Do and G. Rothmel, "On the use of mutation faults in empirical assessments of test case prioritization techniques," *IEEE Trans. Softw. Eng.*, vol. 32, no. 9, pp. 733–752, Sep. 2006.

NAHLAH A. ALRAKBAN received the M.S. degree in computer science from Kent State University, OH, USA. She is currently pursuing the Ph.D. degree in software engineering with the College of Computer and Information Sciences, King Saud University, Riyadh, Saudi Arabia.

She is a Lecturer at the Department of Computer Science, Saudi Electronic University. Her research interests include artificial intelligence, optimization, and software engineering, with a particular focus on intelligent software systems, deep reinforcement learning, and deep learning-based optimization techniques.



MUBARAK ALRASHOUD received the Ph.D. degree in computer science from the Department of Computer Science, Ryerson University, Canada, in 2015. He is currently an Associate Professor with the Department of Software Engineering, College of Computer and Information Sciences, KSU. He had served for more than a decade at the Royal Saudi Air Defense Forces (RSADF). He has served RSADF in many roles, including the Ada and C++ Programmer, a System Analyst, the Software Test Engineer, and the IT Manager. His research interests include software engineering decision support, optimizing the software release planning process, fuzzy decision making, and optimization in a big data environment.



M. ABDULLAH-AL-WADUD (Member, IEEE) received the B.S. degree in computer science and the M.S. degree in computer science and engineering from the University of Dhaka, Bangladesh, in 2003 and 2004, respectively, and the Ph.D. degree in computer engineering from Kyung Hee University, South Korea, in 2009. He was a Faculty Member with the Department of Industrial and Management Engineering, Hankuk University of Foreign Studies, South Korea, from 2009 to 2014.

He was a Lecturer with Daffodil International University, Bangladesh, and East West University, Bangladesh. He is currently an Associate Professor with the Department of Software Engineering, King Saud University, Saudi Arabia. His research interests include artificial intelligence, optimization, computer vision, cloud computing, recommender systems, software engineering, sensors, and ad hoc networks.

...